

OLDIA: Operational Lifecycle Determinism in
Intelligent Architectures
Validated, Falsified, and Deployed at Civilization Scale

Sean Chatman
wasm4pm Research Group

June 2026

Acknowledgments

This work stands on the shoulders of the process mining discipline established by Wil M. P. van der Aalst, whose insistence that conformance between declared models and observed event logs is not a discrepancy but a defect provided the theoretical and methodological foundation for the Object-Level Diagnostic Inference Architecture described here. The van der Aalst constitution—*trust only event evidence that can be mined into a conforming object-centric process*—pervades every layer of this design.

Deep acknowledgment is owed to the original researchers whose work this architecture honors: Edward H. Shortliffe and the MYCIN team, who demonstrated that structured knowledge could reason reliably under uncertainty; Allen Newell and Herbert A. Simon, whose General Problem Solver first articulated the architecture of symbolic search; Lee D. Erman and Victor R. Lesser for Hearsay-II’s blackboard model of cooperative problem solving; Bruce G. Buchanan and Edward A. Feigenbaum for DEN-DRAL’s demonstration that domain knowledge encodes discovery; Joseph Weizenbaum for ELIZA’s uncomfortable lesson about the boundary between language and understanding; John Alan Robinson for the resolution principle that made machine inference tractable; Janet Kolodner, Agnar Aamodt, and Enric Plaza for case-based reasoning; Richard E. Fikes and Nils J. Nilsson for STRIPS; John E. Laird, Allen Newell, and Paul S. Rosenbloom for SOAR; Roger C. Schank for Conceptual Dependency; Karl R. Popper for the logical structure of refutation; Alain Colmerauer and his colleagues for Prolog; Terry Winograd for SHRDLU; Kenneth M. Colby for PARRY; and Patrick H. Winston for structural learning from examples. Each of these systems left a visible seam in OLDIA’s breed taxonomy.

The author is grateful to the open-source communities whose infrastructure makes a deterministic, auditable architecture feasible at the scale described here: the **WebAssembly** Working Group and the contributors to `wasm-bindgen` and `wasm-pack`, whose portable binary format provides the isolation boundary between the inference kernel and its host; the **BLAKE3** team, whose cryptographic hash function supplies the receipt-chain integrity guarantees that make replay and conformance auditing meaningful; and the **Rust** language community, whose ownership model and compilation guarantees eliminate an entire class of runtime defects before any event log is ever written.

Any errors of judgment, gaps in conformance, or violations of the van der Aalst constitution that remain in this work are the sole responsibility of the author.

Contents

Acknowledgments	1
Abstract	1
1 Introduction	3
1.1 The Epistemological Crisis in Deployed AI	3
1.1.1 Theory without operational proof	3
1.1.2 The deployment gap in practice	4
1.2 The Research Questions	4
1.3 Contributions	5
1.3.1 The wasm4pm cognition breed framework	5
1.3.2 The OCEL provability layer	5
1.3.3 The BLAKE3 receipt chain	6
1.3.4 The unfakeable oracle test suite	6
1.3.5 A taxonomy of Fortune-5 deployment patterns	6
1.4 Document Structure	6
1.5 Terminology	7
2 Background and Related Work	9
2.1 The Classical AI Reasoning Systems	9
2.1.1 MYCIN (Shortliffe, 1976)	9
2.1.2 HEARSAY-II (Erman & Lesser, 1980)	10
2.1.3 SOAR (Newell, Laird & Rosenbloom, 1987)	10
2.1.4 Case-Based Reasoning (Kolodner, 1993; Aamodt & Plaza, 1994)	10
2.1.5 Prolog and Robinson Unification (Robinson, 1965; Colmerauer, 1972)	11
2.1.6 STRIPS (Fikes & Nilsson, 1971)	11
2.1.7 GPS (Newell & Simon, 1963)	11
2.1.8 DENDRAL (Buchanan & Feigenbaum, 1978)	11
2.1.9 ELIZA (Weizenbaum, 1966)	12
2.1.10 Autoinstinct Breeds (van der Aalst lineage, 2026)	12
2.2 The Operational Falsifiability Problem	13
2.3 Process Mining and OCEL 2.0	13
2.3.1 Object-Centric Event Logs (OCEL 2.0)	14
2.3.2 DFA Conformance Checking	14
2.3.3 Why Process Mining Is the Natural Audit Framework	14
2.4 WebAssembly as a Deployment Target	15
2.4.1 Determinism Guarantees	15
2.4.2 Build Targets and Toolchain	15

2.5	BLAKE3 and Cryptographic Receipt Chains	15
2.5.1	Receipt Construction	16
2.5.2	Cryptographic Proof of Process	16
2.6	Summary	16
3	The OLDIA Framework: Architecture and Design	18
3.1	Design Principles	18
3.1.1	Operational Falsifiability as a First-Class Property	18
3.1.2	No Wall-Clock Timestamps	18
3.1.3	OCEL Log Inside <code>output_hash</code>	19
3.1.4	Single Choke Point Enforcement	19
3.1.5	Additive-Only <code>ContractResult</code>	19
3.2	The Breed Contract Interface	19
3.2.1	The <code>TraceStep</code> Struct	20
3.2.2	The <code>BreedOutput</code> Struct	21
3.3	The OCEL 2.0 Provability Layer	21
3.3.1	OCEL 2.0 Data Structures	21
3.3.2	Log Derivation	22
3.3.3	Lifecycle Models	22
3.3.4	Conformance Checking	23
3.3.5	Temporal Conformance	23
3.4	The <code>run_breed</code> Enforcement Point	24
3.5	The <code>ContractResult</code> and Receipt Chain	25
3.6	The Determinism Guarantee	26
3.7	The Fidelity Upgrades	27
3.7.1	Hearsay: KSAR Priority Queue	27
3.7.2	SOAR: Bounded Subgoal on Tie Impasse	28
3.7.3	CBR: Full 4R Cycle	28
3.7.4	Prolog: Flat-Term Robinson Unification	29
3.8	Summary	29
4	Falsification Methodology and Test Evidence	31
4.1	The Falsifiability Hierarchy	31
4.1.1	Rank 0: Compilation	31
4.1.2	Rank 1: Mathematical Theorem	31
4.1.3	Rank 2: Domain Contract	32
4.1.4	Rank 3: Operational Invariant	32
4.1.5	Rank 4: Unfakeable Oracle	32
4.2	The Unfakeable Oracle Catalogue	33
4.2.1	MYCIN — CF Strict Boundary (Rank 4)	33
4.2.2	Prolog — Grandparent Derivation (Rank 4)	33
4.2.3	STRIPS — Plan Soundness Simulation (Rank 4)	34
4.2.4	CBR — Jaccard Identity (Rank 4)	34
4.2.5	Hearsay — Opportunistic Order (Rank 4)	35
4.2.6	AutoinstinctLearning — Monotone Distance (Rank 4)	35
4.2.7	GPS — Difference Coverage (Rank 3)	35
4.2.8	DENDRAL — Forbid Absoluteness (Rank 4)	35
4.2.9	SOAR — Antisymmetry (Rank 3)	36

4.2.10	ELIZA — Keystack Priority (Rank 3)	36
4.2.11	AutoinstinctSemantics — ATRANS/PTRANS (Rank 4)	36
4.3	The OCEL Negative Battery	37
4.4	Determinism as a Mathematical Property	37
4.5	The Test Corpus Scale	38
4.5.1	Rust Test File Breakdown	38
4.5.2	TypeScript Test Breakdown	38
4.5.3	Oracle Rank Distribution	39
4.5.4	Timing Evidence	40
4.6	Summary	40
5	Fortune 5 Deployment Patterns at Microsecond Latency	41
5.1	The Operational Economics of Microsecond Reasoning	41
5.2	Pattern 1: Clinical Trial Audit Infrastructure (MYCIN Class)	42
5.3	Pattern 2: Supply Chain Plan Certification (STRIPS Class)	43
5.4	Pattern 3: Regulatory Preference Enforcement (SOAR Class)	44
5.5	Pattern 4: R&D Institutional Memory (CBR Class)	45
5.6	Pattern 5: Customer Signal Fusion (Hearsay Class)	46
5.7	Pattern 6: Quality Defect Hypothesis (DENDRAL Class)	47
5.8	Pattern 7: Contract Intelligence (Prolog Class)	48
5.9	Pattern 8: Goal-Directed Planning Under Constraint (GPS Class)	49
5.10	The Composition Pattern	50
5.11	Summary	50
6	Beyond the Original Vision: Capabilities the Inventors Could Not Have Imagined	52
6.1	The Substrate Was Wrong by Six to Nine Orders of Magnitude	52
6.2	Operational Falsifiability as a New Discipline	53
6.3	Ensemble Reasoning at Operational Tempo	54
6.4	Composition Without Integration Work	54
6.5	Process Mining Over Reasoning Chains	55
6.6	Cryptographic Provenance as Civil Infrastructure	56
6.7	The Speed Enables Real-Time Conformance Checking	56
6.8	The Unfakeable Test Suite as a New Kind of Specification	57
6.9	Summary: A New Epistemic Category	58
7	Conclusion and Future Work	60
7.1	Summary of Contributions	60
7.2	The Central Claim Restated	61
7.3	The Civilization-Scale Claim	62
7.4	Future Work	62
7.5	Closing	63
A	Test Suite Evidence	65
A.1	Complete Test File Inventory	65
A.2	The 13 Lifecycle Models	65
A.3	The Prolog Grandparent Test (Annotated)	67
A.4	The OCEL Conformance Refusal Mechanism	69
A.5	Determinism Properties	70

List of Tables

4.1	Rust test files in <code>crates/wasm4pm-cognition/tests/</code>	39
4.2	Test distribution by oracle rank (655 total)	39
A.1	Complete test file inventory for <code>wasm4pm-cognition</code> (verified 2026-06-09). Oracle rank L1 = known-input or property-based unfakeable oracle; L2 = routing/dispatch coverage; L3 = integration/end-to-end.	65

OLDIA

Operational Lifecycle Determinism in Intelligent Architectures

Validated, Falsified, and Deployed at Civilization Scale

Sean Chatman

wasm4pm Research Group

June 2026

Keywords: classical AI, process mining, OCEL 2.0, WebAssembly, falsifiability, BLAKE3, cognition breeds

Abstract

For seventy years, classical artificial intelligence produced architectures that were theoretically elegant and operationally unverifiable. MYCIN diagnosed infectious disease with demonstrable clinical accuracy yet left no falsifiable execution trail. SOAR modelled universal cognition with formal preference algebras yet offered no mechanism for a downstream auditor to confirm that a particular run obeyed the declared lifecycle. Hearsay, GPS, STRIPS, DENDRAL, ELIZA, CBR, Prolog, and their successors shared the same blind spot: correctness was a property of the design document, not of the deployed runtime. Every execution was, in the van der Aalst sense, a dark process—behaviour that existed but could not be mined, replayed, or refused.

This thesis introduces **wasm4pm cognition breeds**: thirteen classical AI architectures reimplemented as WebAssembly modules under a unified provability contract. The thirteen breeds are MYCIN (Shortliffe–Buchanan certainty-factor algebra), Hearsay (KSAR blackboard with opportunistic scheduling), SOAR (full preference algebra with subgoal impasse resolution), Case-Based Reasoning (4R cycle with Jaccard similarity and BLAKE3 retained-case identity), Prolog (Robinson flat-term unification with bounded SLD resolution), STRIPS (plan-soundness-verified forward chaining), GPS (difference-coverage enforced means–ends analysis), DENDRAL (forbidden-candidate elimination), ELIZA (pattern-action reflection), and four Autoinstinct breeds—Vision, Semantics, Neurosis, and Learning—that express emergent inference over perceptual and symbolic substrates. Each breed is not merely implemented; it is *validated, falsified, and receipt-chained*.

The methodological contribution is an **OCEL 2.0 provability layer** wired at the **run_breed** chokepoint in **wasm.rs**. Every breed execution emits an object-centric event log whose lifecycle phases are checked against a statically declared **BreedLifecycleModel** (a deterministic finite automaton over OCEL 2.0 activity labels) before any output is returned. Non-conforming executions are refused, not merely flagged. The **BLAKE3 receipt chain** extends provability from process to artefact: `output_hash = blake3(BreedOutput)`, `run_id = blake3(breed || output_hash)`, and the OCEL log is embedded inside the output before hashing, so the receipt attests to the process, not just the result. Temporal conformance is enforced by strictly monotone `logical_step` attributes; wall-clock time is excluded from the hash domain to preserve determinism under replay.

The test infrastructure comprises **655 tests with unfakeable oracles**: 480 Rust tests and 175 TypeScript tests, with zero failures recorded. Rust coverage spans adversarial bypass attempts (58 tests), mathematical property invariants (31 tests), formal SOAR/STRIPS/CBR invariants (23 tests), 13-breed bit-exact double-run determinism harnesses (16 tests), and OCEL lifecycle conformance (7 tests), among others. Prolog was rewritten from scratch around Robinson unification and bounded SLD resolution; the canonical grandparent test—`grandparent(?0,?2) :- parent(?0,?1), parent(?1,?2)`—

derives `carol` from a two-step chain and is mathematically unfakeable without shared-variable unification.

Measured execution latency across the thirteen breeds runs from 2 to 50 microseconds on 2026 hardware, compiled to both `nodejs` and browser WASM targets via `wasm-pack`. The system operates as **civilization-scale auditable reasoning infrastructure**: any institution that ingests a cognition breed receipt can independently verify, replay, and mine the execution history as a first-class process artefact. The broader significance is architectural: OLDIA demonstrates that the gap between theoretical AI correctness and operational verifiability is not a fundamental limitation of classical reasoning but an engineering choice—one that can be reversed by placing OCEL provability and receipt-chained falsifiability at the execution boundary of every classical AI architecture deployed in the world.

Chapter 1

Introduction

“The question is not whether a machine can think, but whether we can *prove* it thought correctly while it was doing so.”

— paraphrase of the operational falsifiability problem

Classical artificial intelligence produced architectures of genuine intellectual beauty. MYCIN’s certainty-factor algebra, Hearsay-II’s blackboard, STRIPS’s precondition–effect formalism, and SOAR’s preference-based decision cycle each resolved a hard theoretical problem. Yet every one of these systems shipped as a black box. The 1976 MYCIN deployment had no audit trail. Hearsay-II’s knowledge-source activation record was discarded at runtime. A correct diagnosis and an incorrect one looked identical from outside the process. The epistemological gap between *theoretical soundness* and *operational verifiability* was, for forty years, accepted as a fact of life.

This thesis argues that the gap is no longer necessary and that closing it changes the economics of deploying certified reasoning at Fortune-5 scale.

1.1 The Epistemological Crisis in Deployed AI

1.1.1 Theory without operational proof

Shortliffe and Buchanan’s certainty-factor formula ? is mathematically well-defined. Given two independent evidence items e_1 and e_2 and a hypothesis h , the combined certainty factor obeys:

$$\text{CF}(h, e_1 \wedge e_2) = \begin{cases} \text{CF}_1 + \text{CF}_2 (1 - \text{CF}_1) & \text{if both positive} \\ \text{CF}_1 + \text{CF}_2 (1 + \text{CF}_2) & \text{if both negative} \\ \frac{\text{CF}_1 + \text{CF}_2}{1 - \min(|\text{CF}_1|, |\text{CF}_2|)} & \text{otherwise} \end{cases}$$

The formula is sound. What it does not provide is any mechanism for an external auditor to verify, after the fact, that the running system applied this formula to the correct inputs in the correct order, that no rule was silently skipped, and that the final recommendation was derived from a process consistent with the declared model.

This is not a criticism of Shortliffe and Buchanan. Runtime process provability was not a design goal in 1976 because there was no infrastructure to support it. The insight of this thesis is that such infrastructure now exists — in the form of object-centric event logs, WebAssembly’s deterministic execution model, and cryptographic receipt chains — and that retrofitting it onto classical architectures reveals both their robustness and the depth of the original design decisions.

1.1.2 The deployment gap in practice

The gap between theory and deployment shows up in three ways that matter to practitioners.

Silent deviation. A rule-based system that has been modified, patched, or ported may execute a different process than its specification describes. Without a conformance check, operators cannot distinguish a correct execution from a deviant one. Van der Aalst’s work on process mining [1] formalised this problem for business processes; this thesis applies the same discipline to AI reasoning architectures.

Unstable audit trails. Regulated industries (finance, pharmaceutical, critical infrastructure) require audit evidence that a decision process conformed to a declared model. Classical AI systems produce result logs, not process logs. A result log proves that a recommendation was emitted; it does not prove that the recommendation was reached by the declared reasoning process.

Unverifiable latency claims. Microsecond-latency AI is increasingly demanded in high-frequency trading, real-time threat detection, and autonomous systems. A system that achieves low latency by skipping reasoning steps — rather than by executing them efficiently — is operationally indistinguishable from one that does not, unless the execution process is independently verified.

All three problems share a common structure: the *output* is observable, but the *process* that produced it is not. This thesis proposes and validates a framework that makes the process observable, falsifiable, and cryptographically attested.

1.2 The Research Questions

Three research questions organise this investigation.

RQ1 *Can classical AI reasoning be made operationally falsifiable — not just tested at development time, but continuously proven conforming at runtime?*

This question requires distinguishing two kinds of evidence. A test suite proves that the implementation behaved correctly on a finite set of inputs under controlled conditions. Operational falsifiability means that, for every execution in production, an independent artifact — the OCEL event log and its associated BLAKE3 receipt — can be presented to an auditor who had no prior knowledge of the system and who can verify, from that artifact alone, that the execution followed the declared lifecycle model. Chapter ?? presents the formal model; Chapter ?? presents evidence from 655 tests that the implementation meets the specification.

RQ2 *Does operational falsifiability change the deployment economics for Fortune-5 enterprises?*

The hypothesis is that certified reasoning — reasoning for which conformance evidence is continuously produced and cryptographically attested — commands a qualitatively different position in the enterprise procurement conversation than uncertified reasoning of equal accuracy. Chapter ?? develops a taxonomy of eight deployment patterns that become commercially accessible when reasoning latency drops below 50 microseconds and every execution produces a signed receipt. The chapter argues that the combination of latency and provability unlocks markets that are currently closed to AI: real-time regulatory compliance, intraday risk certification, and autonomous systems with insurance-grade audit trails.

RQ3 *What becomes possible at 2–50 microsecond latency that the original inventors could not have imagined?*

MYCIN was a batch consultation system. Hearsay-II processed speech in near-real-time on 1970s hardware. Neither system was designed for continuous, high-frequency production deployment. When the same reasoning architectures execute in 2–50 microseconds inside a WebAssembly sandbox, new application classes emerge: per-transaction fraud scoring with interpretable rule traces, per-packet network policy reasoning with signed receipts, and per-event process monitoring that emits OCEL conformance reports faster than the underlying business process can advance. Chapter ?? catalogues these patterns and the conditions under which they are achievable.

1.3 Contributions

This thesis makes five original contributions.

1.3.1 The wasm4pm cognition breed framework

The central artifact of this thesis is a collection of thirteen validated, operationally falsifiable implementations of classical AI architectures, compiled to WebAssembly and exposed through a uniform `cognition_run` interface. The thirteen *breeds* are: MYCIN (certainty-factor production rules), Hearsay-II (blackboard with KSAR priority queue), SOAR (preference-algebra decision cycle with chunking), CBR (case-based reasoning with full 4R cycle), Prolog (Robinson unification with bounded SLD resolution), STRIPS (precondition–effect planning with soundness verification), GPS (general problem solver with means–ends analysis), DENDRAL (hypothesis generation with forbidden-candidate elimination), ELIZA (pattern-directed dialogue), and four Autoinstinct breeds (vision, semantics, neurosis, learning).

Each breed is implemented from primary sources — Shortliffe and Buchanan for MYCIN, Newell for SOAR and GPS, Fikes and Nilsson for STRIPS, and so on — and validated against mathematical invariants that rule out stub implementations. The Prolog breed, for example, must derive `grandparent(?0,?2)` from `parent(?0,?1) :- ...` chains using genuine shared-variable unification; no pattern-matching shortcut can satisfy this test. The full test suite comprises 655 tests (480 Rust, 175 TypeScript), all passing as of 2026-06-09.

1.3.2 The OCEL provability layer

Every breed execution produces an OCEL 2.0 ? event log capturing the reasoning trace as a sequence of typed events over named objects. Thirteen lifecycle models — one per breed — define the lawful sequences of event types for that architecture. A DFA-based conformance checker (`validate_ocel_alignment`) verifies, at the execution choke point in `wasm.rs`, that every emitted trace conforms to the declared model before the result is returned to the caller. Non-conforming executions are refused, not logged for later review.

The OCEL layer is implemented in `crates/wasm4pm-cognition/src/ocel.rs` (632 lines). Logical-step attributes rather than wall-clock timestamps ensure that conformance evidence is independent of hardware timing and replayable in any environment.

1.3.3 The BLAKE3 receipt chain

Every successful execution emits a cryptographic receipt. The `output_hash` is `blake3(serde_json(BreedOutput))`, where `BreedOutput` contains the OCEL log. The receipt therefore attests to *process*, not just result: an auditor who holds the receipt can verify that the output was produced by a specific reasoning trace, not merely that some output was produced. The `run_id` is `blake3(breed || output_hash)`, making receipts cross-breed comparable. The `replay_pointer` (the first 16 hex characters of `output_hash`) provides a compact, stable identifier suitable for embedding in external audit systems.

1.3.4 The unfakeable oracle test suite

The 655-test suite is designed so that no stub implementation — no implementation that returns hardcoded outputs without executing the declared algorithm — can pass. This property, which we call *oracle unfakeability*, is achieved by testing mathematical invariants rather than specific output values. MYCIN tests the strict boundary at CF = 0.2 (rejected) and CF = 0.201 (accepted), which requires the continuous formula to be evaluated. CBR tests that identical query and case produce Jaccard similarity of exactly 1.0. SOAR tests that circular worse-than preferences produce deterministic termination, which requires the full preference algebra to execute. AutoinstinctLearning tests that plan-step distance to goal is non-increasing, which requires genuine planning.

1.3.5 A taxonomy of Fortune-5 deployment patterns

Chapter ?? presents eight deployment patterns enabled by the combination of microsecond latency and operational provability. The patterns are grounded in the verified timing characteristics of the breed framework (2–50 microseconds per execution on 2026 hardware) and in the structural requirements of the OCEL provability and BLAKE3 receipt layers. Each pattern is described in terms of the reasoning architecture it employs, the conformance evidence it produces, and the commercial context in which it is valuable.

1.4 Document Structure

The remainder of this thesis is organised as follows.

Chapter 2 reviews the classical AI architectures that form the substrate of the breed framework, the process mining literature that motivates the OCEL provability layer, and prior work on certified AI and runtime verification.

Chapter ?? describes the thirteen breed implementations in detail: their theoretical foundations, their Rust/WASM implementations, and the design decisions that make them simultaneously faithful to the originals and amenable to operational falsifiability.

Chapter ?? presents the OCEL provability layer: the object-centric event log schema, the thirteen lifecycle models, the DFA conformance checker, and the temporal conformance enforcement mechanism.

Chapter ?? presents the BLAKE3 receipt chain: its construction, its relationship to the OCEL log, and its properties as an audit instrument.

Chapter ?? presents the validation evidence: the 655-test suite, the oracle unfakeability argument, and the timing measurements that establish the microsecond-latency claims.

Chapter ?? presents the taxonomy of Fortune-5 deployment patterns and argues that operational falsifiability is a necessary condition for their commercial viability.

Chapter 7 summarises the contributions, states the limitations of the current implementation, and identifies directions for future work.

1.5 Terminology

The following terms are used with precise meanings throughout this thesis.

Breed. A *breed* is a validated, operationally falsifiable implementation of a classical AI reasoning architecture, compiled to WebAssembly and exposed through the uniform `cognition_run` interface. The term *breed* is preferred over *algorithm* or *agent* because each implementation encompasses both the reasoning procedure and the lifecycle model against which its execution is continuously verified. Thirteen breeds are implemented in this thesis.

OCEL. *OCEL* (Object-Centric Event Log) ? is a log format in which events are related to multiple objects of multiple types, rather than to a single case identifier. OCEL 2.0 extends the original format with typed attributes on event-object relationships. This thesis uses OCEL 2.0 as the provability substrate: each breed execution emits an OCEL log in which the reasoning objects (rules, knowledge sources, goals, cases) are first-class entities with traceable lifecycles.

Lifecycle model. A *lifecycle model* is a finite automaton that defines the lawful sequences of event types for a given breed. For example, the MYCIN lifecycle model requires that every execution begin with `rule_evaluate` events, followed by `cf_combine` events, and conclude with a `threshold_check` event. A trace that omits the `threshold_check` or emits events in the wrong order is non-conforming and is refused at the execution choke point.

Conformance. *Conformance* is the binary property of an OCEL trace with respect to a lifecycle model: a trace either conforms or it does not. This thesis does not use graded conformance metrics (fitness, precision) in the execution-time path; those metrics appear in the validation chapter where the test suite is analysed. At the choke point, conformance is a gate: non-conforming traces are refused.

Receipt chain. The *receipt chain* is the sequence of BLAKE3-hashed receipts produced by successive executions of a breed. Each receipt’s `output_hash` covers the OCEL log, making the chain a tamper-evident record of the reasoning process, not merely of the results. The term *chain* reflects the structural dependency: `run_id` is derived from `output_hash`, which is derived from the `BreedOutput`, which contains the OCEL log.

Unfakeable oracle. An *unfakeable oracle* is a test assertion that can only be satisfied by an implementation that executes the declared algorithm. It is distinct from a value-equality assertion (which tests a specific output) and from a property-based assertion (which tests a statistical distribution of outputs). An unfakeable oracle tests a mathematical invariant that is *structurally impossible* to satisfy without executing the algorithm: for example, the Jaccard identity invariant for CBR, or the shared-variable unification requirement for Prolog.

KSAR. A *KSAR* (Knowledge Source Activation Record) is the scheduling primitive of the Hearsay-II blackboard architecture ?. When a knowledge source detects a triggering condition on the blackboard, it creates a KSAR recording its identity, the triggering evidence, and an estimated priority (`rating = certainty * trigger_cf`). The blackboard scheduler selects the highest-rated pending KSAR for execution. In the wasm4pm implementation, KSARs are managed in a priority queue using `f32::total_cmp` for deterministic ordering, and stale KSARs are re-rated when the blackboard state changes.

Chapter 2

Background and Related Work

This chapter surveys the intellectual heritage of the thirteen reasoning breeds implemented in OLDIA, establishes the theoretical gap that motivates the research, and reviews the three technical pillars on which the implementation rests: process mining with OCEL 2.0, WebAssembly as a deterministic deployment substrate, and BLAKE3 cryptographic receipt chains.

2.1 The Classical AI Reasoning Systems

The thirteen breeds instantiated in OLDIA span five decades of AI research, from the earliest production-rule systems of the 1960s to the autoinstinct breeds introduced in 2026. Each breed is characterised by a distinct computational model, a set of mathematical invariants, and a body of domain-specific theory. The following subsections cover historical context, original authorship, and the core algorithmic insight for each breed.

2.1.1 MYCIN (Shortliffe, 1976)

MYCIN ? was developed at Stanford as a clinical decision-support system for antibiotic selection in bacterial infections. Its central contribution is the *Certainty Factor* (CF) algebra, a non-probabilistic calculus for combining evidence from production rules.

The combination formula distinguishes three sign cases. When both evidence values a and b are positive:

$$\text{CF}(a, b) = a + b - ab \quad (2.1)$$

When both are negative:

$$\text{CF}(a, b) = a + b + ab \quad (2.2)$$

When the signs differ:

$$\text{CF}(a, b) = \frac{a + b}{1 - \min(|a|, |b|)} \quad (2.3)$$

A strict threshold of $\text{CF} > 0.2$ is required before a hypothesis advances. The boundary is continuous and unfakeable: in the OLDIA test suite a CF of exactly 0.200 is rejected while 0.201 is accepted (`tests/breed_adversarial.rs`, `test mycin_cf_boundary`). Equation 2.1 expresses bounded Dempster–Shafer-style fusion: positive evidence accumulates sublinearly, preventing runaway certainty.

2.1.2 HEARSAY-II (Erman & Lesser, 1980)

HEARSAY-II ? was built for the ARPA Speech Understanding Research project. Its architectural innovation is the *blackboard*: a shared, structured working memory on which independent *Knowledge Sources* (KSs) collaborate without direct coupling.

Each KS registers *Knowledge Source Activation Records* (KSARs). An opportunistic scheduler selects the highest-rated KSAR at each cycle:

$$\text{rating} = \text{ks_certainty} \times \text{trigger_cf} \quad (2.4)$$

Hypothesis fusion uses a *noisy-OR* model: independent supporting hypotheses combine multiplicatively on the failure side, so joint support = $1 - \prod_i (1 - p_i)$. The unfakeable scheduling invariant verified in OLDIA is that a higher-rated KSAR always fires before a lower-rated one regardless of declaration order (`hearsay_opportunistic_order`). The implementation (`hearsay.rs`, 584 lines) uses `f32::total_cmp` for deterministic priority-queue ordering and re-rates stale KSARs when blackboard state changes.

2.1.3 SOAR (Newell, Laird & Rosenbloom, 1987)

SOAR ? is a unified cognitive architecture motivated by Newell’s call for a single computational model of mind. Its operator selection mechanism uses a *preference algebra* with six relations: REQUIRE, PROHIBIT, BETTER-THAN, WORSE-THAN, BEST, and INDIFFERENT.

When preferences produce a *tie impasse*—no operator is strictly preferred—SOAR creates a subgoal to resolve the impasse. In OLDIA the subgoal depth is bounded to two to guarantee termination; this matches the standard bounded-rationality assumption in the SOAR literature. The antisymmetry invariant—circular WORSE-THAN preferences produce deterministic termination rather than infinite regress—is verified in `soar_antisymmetry`. The implementation (`soar.rs`, 675 lines) outputs `chunk.pref` records that expose the operator selection trace for process-mining audit.

2.1.4 Case-Based Reasoning (Kolodner, 1993; Aamodt & Plaza, 1994)

Case-Based Reasoning (CBR) ?? derives solutions by analogy with past cases. The *4R cycle* is the canonical process model: **Retrieve**, **Reuse**, **Revise**, **Retain**.

OLDIA implements all four stages. Retrieval uses Jaccard similarity indexed over a `BTreeMap` for deterministic ordering. Reuse applies substitutional adaptation. Revise validates the adapted solution against domain constraints. Retain selects high-quality cases for storage, assigning each a BLAKE3-derived case identifier to guarantee stable cross-run identity. The identity invariant—identical query and case produce Jaccard similarity exactly 1.0—is verified in `cbr_jaccard_identity`.

2.1.5 Prolog and Robinson Unification (Robinson, 1965; Colmerauer, 1972)

Robinson’s *resolution principle* ? established that first-order theorem proving is reducible to a single inference rule applied to clauses in conjunctive normal form. Colmerauer’s Prolog ? operationalised this as SLD resolution over Horn clauses, producing the first practical logic programming language.

OLDIA implements flat-term Robinson unification with positional variables written ?N. The unification kernel (`prolog8/kernel.rs`, 1143 lines) supports bounded SLD resolution to prevent divergence. The canonical unfakeable test is the grandparent derivation:

```
grandparent(?0,?2) :- parent(?0,?1), parent(?1,?2).
```

Given facts `parent(alice,bob)` and `parent(bob,carol)`, the system must derive `grandparent(alice,carol)`. This query is unfakeable without shared-variable unification: pattern matching alone cannot chain through the intermediate variable ?1.

2.1.6 STRIPS (Fikes & Nilsson, 1971)

STRIPS ? introduced the *add/delete list* representation for planning operators and the *frame axiom* convention: an action changes only what its lists explicitly specify. The planner searches forward from an initial state using iterative deepening.

Two formal properties define STRIPS correctness. *Plan soundness* requires that every action’s preconditions are satisfied in the state at the point of application. *Plan completeness* requires that all goal literals are true in the final state. OLDIA verifies both properties at runtime: a simulated execution walks the plan step-by-step, verifying each precondition and confirming goal satisfaction (`strips_plan_soundness`).

2.1.7 GPS (Newell & Simon, 1963)

The General Problem Solver ? introduced *means-ends analysis*: an agent selects operators by comparing the current state with a goal state, identifying the most significant difference, and applying an operator that reduces that difference. A *difference table* maps difference types to applicable operators.

Operator selection in OLDIA uses fewest unmet preconditions as the heuristic. The unfakeable invariant is *difference reduction monotonicity*: each plan step must reduce or maintain the number of unmet goal facts—a plan step that increases the difference count is a formal defect. GPS also requires *difference coverage*: the plan must address every unmet goal fact (`gps_difference_coverage`).

2.1.8 DENDRAL (Buchanan & Feigenbaum, 1978)

DENDRAL ? was the first expert system to match domain expert performance, deriving molecular structures from mass spectrometry data. Its inference model is *hypothesis generation with hard constraints*: a generate-and-test loop eliminates candidates that violate forbidden structural patterns.

The *forbid* constraint is absolute in OLDIA. A candidate matching a forbidden pattern is eliminated regardless of its plausibility score: the highest-scoring candidate in the enumerated set is still removed if it is forbidden (`dendral_forbid_absolute`). This is the key distinction from soft-constraint optimisation: forbid is a monotone elimination rule, not a penalty.

2.1.9 ELIZA (Weizenbaum, 1966)

ELIZA ? demonstrated that a simple pattern-transformation system could create the illusion of understanding. Its mechanism is a *keystack*: patterns are ordered by specificity priority, and the highest-priority matching pattern triggers a response transformation.

Three invariants are verified in OLDIA. *Specificity ordering* requires that a higher-priority pattern fires before lower-priority alternatives. *Pronoun reflection* requires that first-person pronouns in the input are reflected correctly in the response. *Catch-all guarantee* requires that the system always produces a response—there is no input for which the keystack returns empty.

2.1.10 Autoinstinct Breeds (van der Aalst lineage, 2026)

Four autoinstinct breeds extend the classical canon with implementations drawn from the same 1960s–1970s AI lineage.

AutoinstinctLearning. Inspired by STRIPS/HACKER ?, this breed implements bit-wise heuristic planning. The unfakeable mathematical invariant is *monotone distance reduction*: the Hamming-like distance to the goal state must be non-increasing across plan steps. A plan that permits backsliding is provably incorrect.

AutoinstinctNeurosis. Based on Colby’s PARRY affect machine ?, this breed models belief-conflict processing and paranoia. Affect state transitions are governed by belief conflicts detected in the input; the conflict severity determines the strength of the paranoid response.

AutoinstinctVision. Derived from Winograd’s SHRDLU Blocks World perception ?, this breed implements symbolic scene understanding. The key data structures are the *supported-by* chain (which block rests on which) and the *clear-object* predicate (a block is clear if nothing rests on it).

AutoinstinctSemantics. Based on Schank’s Conceptual Dependency theory ?, this breed maps natural language utterances to primitive acts: ATRANS (abstract transfer), PTRANS (physical transfer), and MTRANS (mental transfer). Semantic coverage—every input clause maps to at least one primitive act—is the core invariant.

2.2 The Operational Falsifiability Problem

Popper’s falsifiability criterion ? holds that a scientific claim is meaningful only if it is in principle refutable by observation. Applied to AI reasoning systems, the criterion demands that claims about reasoning behaviour be refutable by examining what the system actually does at runtime.

The thirteen classical systems surveyed above were theoretically falsifiable: each had a documented algorithmic model against which behaviour could in principle be compared. MYCIN’s certainty factor algebra was published in mathematical detail. SOAR’s preference semantics were formally specified. STRIPS plans were structurally verifiable. Yet in practice these systems were *operationally opaque*: there was no infrastructure for recording, replaying, or auditing the actual sequence of reasoning steps taken during a run. A clinician using MYCIN could see its conclusions but could not independently verify that the CF combination formula had been applied correctly, that the threshold had been honoured, or that no rule had fired out of order.

This gap between theoretical falsifiability and operational transparency is what we term the *operational falsifiability problem*. It has three dimensions. First, *execution opacity*: the internal reasoning trace is not recorded in any durable, inspectable form. Second, *model-log divergence*: even when some logging exists, there is no systematic method for comparing the logged behaviour against the declared process model. Third, *cryptographic unattestedness*: there is no proof that a logged trace corresponds to a specific input/output pair, so logs can be altered or fabricated after the fact.

The absence of audit infrastructure was not merely a practical inconvenience; it was an epistemological deficiency. Without durable, cryptographically attested, process-mined event logs, claims that a system “behaved correctly” reduce to assertions rather than proofs. OLDIA addresses all three dimensions: it records every reasoning step as an OCEL 2.0 event, validates the log against a declared DFA lifecycle model at the run choke point, and attests the entire output—including the event log—with a BLAKE3 receipt.

2.3 Process Mining and OCEL 2.0

Process mining ? is the discipline of extracting process knowledge from event logs recorded by information systems. It occupies the intersection of data mining, machine learning, and formal process modelling. The three core activities are *process discovery* (learning a process model from a log), *conformance checking* (measuring the fit between a log and a declared model), and *enhancement* (improving an existing model using log evidence).

Conformance checking is evaluated against four quality dimensions introduced by van der Aalst. *Fitness* measures the fraction of log traces that are replayable on the process model. *Precision* measures the fraction of model behaviour that is actually observed in the log (a permissive model that allows arbitrary behaviour scores low precision). *Generalization* measures how well the model explains unseen traces. *Simplicity* penalises unnecessarily complex models. The fitness threshold of > 0.85 is the standard gate for a valid process model in the wasm4pm ecosystem.

2.3.1 Object-Centric Event Logs (OCEL 2.0)

Classical process mining assumes a flat, case-centric log: each event belongs to exactly one case. This assumption fails for AI reasoning systems, in which a single run may simultaneously evolve multiple objects—a rule, a hypothesis, a case, a plan, and an operator are distinct objects with independent lifecycles that interact during a single episode.

OCEL 2.0 ? generalises the event log to the *object-centric* setting. An OCEL event references a set of objects rather than a single case identifier, and each object type carries its own lifecycle model. The key data structures are:

- **OcelEvent**: event identifier, activity name, logical timestamp, map from object type to object identifier list.
- **OcelObject**: object identifier, object type, attribute map.
- **OcelLog**: set of events, set of objects, object-type map.

OLDIA derives an OCEL log from every breed run via the `derive_ocel(breed_id, run_id, &[TraceStep])` function (`src/ocel.rs`, 632 lines). Timestamps are *logical steps* rather than wall-clock times, eliminating non-determinism from concurrency or scheduling jitter.

2.3.2 DFA Conformance Checking

Each of the thirteen breeds has a declared *lifecycle model*: a deterministic finite automaton (DFA) specifying the legal sequence of reasoning phases. For example, the MYCIN lifecycle is `init` $\rightarrow$ `rule_eval` $\rightarrow$ `cf_combine` $\rightarrow$ `threshold` $\rightarrow$ `conclude`. The CBR lifecycle enforces the 4R ordering: `retrieve` $\rightarrow$ `reuse` $\rightarrow$ `revise` $\rightarrow$ `retain`.

The function `validate_ocel_alignment(log, model)` implements a DFA phase checker: it replays the event sequence of each object against the breed’s DFA, returning a `ConformanceResult` that includes fitness, the set of non-conforming objects, and a phase-violation trace. `check_temporal_conformance(log)` additionally verifies that logical steps are strictly monotone—no event may carry the same or smaller step index as a preceding event for the same object.

Both checks are wired at the `run_breed` choke point in `wasm.rs`: a non-conforming execution is refused at the boundary rather than allowed to produce an attested but invalid receipt.

2.3.3 Why Process Mining Is the Natural Audit Framework

Process mining is not merely a useful tool for auditing AI reasoning systems; it is the *natural* framework for this task, for three reasons.

First, reasoning systems are inherently process-structured: they move through phases (hypothesis generation, evidence combination, preference resolution, goal reduction) in a declared order. Process mining is designed exactly to verify whether observed event sequences conform to declared process models.

Second, OCEL 2.0 captures the multi-object nature of AI reasoning. A single MYCIN run involves multiple rules, each with its own CF, and multiple hypotheses, each with their own lifecycle. A flat event log would collapse this structure; OCEL preserves it.

Third, the van der Aalst fitness/precision/generalization/simplicity framework provides quantitative metrics that translate directly into pass/fail criteria for automated CI gates. A breed that achieves fitness < 0.85 fails the conformance gate and cannot produce an attested receipt.

2.4 WebAssembly as a Deployment Target

WebAssembly (WASM) ? is a stack-based virtual instruction set designed for safe, portable, near-native-speed execution. It was standardised by the W3C in 2019 and is now supported natively in all major browsers and Node.js runtimes.

2.4.1 Determinism Guarantees

WASM is *deterministic by specification*: given the same module, the same linear memory initialisation, and the same sequence of host function calls, the execution produces bit-identical output on every platform. This property is essential for OLDIA. Determinism

is a merge gate in the `wasm4pm` project: the `breed_determinism.rs` test suite runs each of the thirteen breeds twice on the same input and compares outputs with bit-exact equality. Any non-determinism—from unordered `HashMap` iteration, floating-point platform variance, or unseeded random number generation—is a defect that blocks merge.

The `to_js_str()` function in `utilities.rs` is the canonical WASM boundary serialiser; `to_js()` silently returns `{}` on `wasm32` targets and is forbidden for structured output.

2.4.2 Build Targets and Toolchain

OLDIA uses `wasm-pack` to compile the Rust crate `wasm4pm-cognition` to both the `nodejs` target (for CI and server-side use) and the `browser` target (for in-browser deployment). The one-time compilation cost is approximately 21 seconds on 2026 hardware. Once compiled, breed execution typically completes in 2–50 microseconds, making WASM-hosted AI reasoning competitive with native execution for interactive applications.

The TypeScript monorepo (`packages/`) wraps the WASM boundary via the `@wasm4pm/kernel` package, which loads the compiled module through `WasmLoader`—a singleton that must be explicitly reset between test runs to prevent state leakage.

2.5 BLAKE3 and Cryptographic Receipt Chains

BLAKE3 [?] is a cryptographic hash function that is simultaneously faster than MD5, cryptographically secure (preimage resistance, second-preimage resistance, collision resistance), and deterministic across platforms. It supports tree-based parallelism and produces 256-bit digests.

2.5.1 Receipt Construction

OLDIA constructs receipts as follows. After a breed run completes, the `BreedOutput` struct (which contains the reasoning result, the OCEL log, and metadata) is serialised to canonical JSON via `serde_json`. The BLAKE3 hash of this serialisation is the `output_hash`:

$$\text{output_hash} = \text{BLAKE3}(\text{serde_json}(\text{BreedOutput})) \quad (2.5)$$

Because `BreedOutput` contains the OCEL log, the `output_hash` is simultaneously a receipt of the reasoning result *and* a receipt of the process trace. A verifier who holds the input and the receipt can independently reproduce the hash; any divergence indicates either a different input or a different execution.

The `run_id` is derived from the breed identifier and the output hash:

$$\text{run_id} = \text{BLAKE3}(\text{breed} \parallel \text{output_hash}) \quad (2.6)$$

The `replay_pointer` is the first 16 characters of the `output_hash`, providing a short human-readable identifier for log correlation.

2.5.2 Cryptographic Proof of Process

The receipt chain closes the operational falsifiability problem. Theoretical falsifiability (the algorithm is documented) plus process conformance (the event log conforms to the declared DFA) plus cryptographic attestation (the BLAKE3 receipt binds the log to the specific input/output pair) yields what we term *operational provability*: a third party can verify, without access to the running system, that a specific input was processed by a specific breed according to its declared lifecycle, and that the stated output was produced.

This is the epistemological foundation of OLDIA. The BLAKE3 receipt does not merely record that something happened; it constitutes evidence that a lawful process happened. In the terminology of the van der Aalst constitution: if the receipt exists and the OCEL log it covers is conformant, then it worked.

2.6 Summary

This chapter has established the intellectual genealogy of the thirteen OLDIA breeds, identified the operational falsifiability problem that motivates the research, and reviewed the three technical pillars of the implementation.

The classical AI systems from 1963 to 1993—GPS, STRIPS, MYCIN, HEARSAY-II, SOAR, CBR, Prolog, DENDRAL, ELIZA—were theoretically falsifiable but lacked the audit infrastructure needed for operational proof. The four autoinstinct breeds extend this lineage while inheriting the same gap. OCEL 2.0 process mining provides the natural audit framework: object-centric event logs capture multi-object reasoning lifecycles, DFA conformance checking verifies that declared phase orderings were honoured, and the fitness/precision metrics provide quantitative pass/fail gates. WebAssembly guarantees bit-exact determinism across platforms, and BLAKE3 receipt chains cryptographically attest that a specific input produced a specific conformant execution. Together these three pillars transform “the code says it worked” into “the event log proves a lawful process happened.”

Chapter ?? describes how these elements are combined in the OLDIA architecture.

Chapter 3

The OLDIA Framework: Architecture and Design

This chapter presents the technical architecture of OLDIA—the Object-Centric Logical Determinism Invariant Architecture. OLDIA is not a testing framework layered atop the cognition layer; it is the cognition layer’s structural law. Every breed execution is governed by a single choke point, every receipt attests to the process that produced it, and every claim of correct behavior is falsifiable by replaying the attached OCEL 2.0 log through a lifecycle conformance checker. The chapter describes each architectural element in turn, from the foundational design principles to the algorithm-level fidelity upgrades that made the OCEL provability layer non-trivial to satisfy.

3.1 Design Principles

Five principles govern every design decision in OLDIA. Violating any one of them collapses the operational falsifiability guarantee.

3.1.1 Operational Falsifiability as a First-Class Property

A system is operationally falsifiable if, and only if, its claimed behavior can be refuted by examining its own artifacts without access to the source code or any external oracle. In OLDIA, the artifact is the `ContractResult` returned by `cognition_run`. The `output_hash` field is a BLAKE3 digest of the serialized `BreedOutput`, which includes the `ocel_log` field. Any consumer who possesses the `ContractResult` can re-hash the output and verify that the OCEL log inside it was not substituted after the fact. Any process mining tool that speaks OCEL 2.0 can replay the log through a lifecycle model and verify that the phases executed in lawful order. Falsifiability is therefore operational: it requires only the output, not the code.

3.1.2 No Wall-Clock Timestamps

Wall-clock timestamps are a determinism hazard. If execution time appears anywhere in the output, two runs of the same breed on the same input will produce different `output_hash` values, breaking the determinism gate. All `OcelEvent` timestamps in OLDIA are set to the constant epoch value "1970-01-01T00:00:00Z". Temporal ordering is encoded in the `logical_step` attribute—a monotonically increasing integer

assigned at event creation time. The conformance checker enforces strict monotonicity of `logical_step`; any duplicate or retrograde value causes conformance refusal. The result is a log that is simultaneously OCEL 2.0 compliant, temporally ordered, and bit-exact across runs.

3.1.3 OCEL Log Inside `output_hash`

The OCEL log is not a side-channel artifact appended after hashing. It is a field of the `BreedOutput` struct, serialized as part of the value that `blake3` digests. The consequence is non-negotiable: the receipt attests to the process, not merely the result. A breed that produced the correct final answer via a non-conforming execution path would produce a different `output_hash` than one that produced the same answer via the lawful path—because the OCEL logs would differ. There is no way to produce a valid receipt for a correct result without a conforming process.

3.1.4 Single Choke Point Enforcement

All thirteen breed implementations are dispatched through a single function: `run_breed` in `wasm.rs`. There is no alternate path. The function signature accepts a breed identifier and a `BreedInput`; it returns a `ContractResult` or an error. Between those two points, in this order, it calls: precondition check, `run()`, postcondition check (including the empty-trace fraud guard), `derive_ocel`, `validate_ocel_alignment`, conformance refusal if fitness is below 1.0, OCEL attachment, BLAKE3 hashing, and result construction. No breed can bypass any of these steps. A breed that attempts to return early from `run()` with an empty trace will be refused at the postcondition check before OCEL derivation is even attempted.

3.1.5 Additive-Only `ContractResult`

The `ContractResult` struct has six frozen fields: `status`, `breed`, `run_id`, `output_hash`, `replay_pointer`, `options_profile`, and `output`. These fields are the external contract between the WASM boundary and every TypeScript consumer in the eleven-package monorepo. They are additive-only: new fields may be appended in future versions, but existing fields are never renamed, removed, or retyped. The `cognition-contract-guard.sh` hook enforces this at commit time. Any change to the frozen field set triggers a build failure with a diagnostic citing the specific violated contract. The `conformance` sub-object (carrying `fitness`, `model_id`, and `refusals`) was added as an additive extension and does not alter the frozen core.

3.2 The Breed Contract Interface

Each of the thirteen breeds implements the `CognitionBreed` trait defined in `breeds/mod.rs`. The trait has four required methods:

```
pub trait CognitionBreed: Send + Sync {
    fn id(&self) -> BreedId;
    fn preconditions(&self, input: &BreedInput)
        -> Result<(), String>;
```

```

fn run(&self, input: &BreedInput)
    -> Result<BreedOutput, BreedError>;
fn postconditions(&self, output: &BreedOutput)
    -> Result<(), String>;
}

```

The `id()` method returns a `BreedId`—a string identifier unique within the registry. The `preconditions` method validates the input before any computation begins; typical checks include required field presence, non-empty fact sets, and rule-count minimums. The `run` method performs the breed-specific inference and returns a `BreedOutput`. The `postconditions` method validates the output after inference; the most important post-condition across all breeds is the empty-trace fraud guard:

```

if output.inference_trace.is_empty() {
    return Err("postcondition: empty inference_trace".into());
}

```

This guard prevents a pathological breed implementation from returning a structurally valid output while having performed no observable reasoning steps. An empty trace would produce an OCEL log containing only the synthetic run-start and run-end events, which no lifecycle model accepts as conforming.

3.2.1 The TraceStep Struct

The atomic unit of observable reasoning is the `TraceStep`:

```

pub struct TraceStep {
    pub step:    usize,
    pub kind:    String,
    pub detail:  String,
    pub depth:   u32,
    pub objects: Vec<(String, String)>,
}

```

The `step` field is a monotonically increasing counter assigned by the breed implementation. The `kind` field encodes the lifecycle phase event type (e.g., "load-fact", "fire-rule", "decision"). The `detail` field carries human-readable content such as the rule identifier fired or the hypothesis posted. The `depth` field records subgoal nesting depth for breeds that support recursive decomposition (STRIPS, GPS, SOAR). The `objects` field carries a vector of (`object_type`, `object_id`) pairs that link the event to the OCEL objects it involves.

3.2.2 The BreedOutput Struct

`BreedOutput` carries three fields that are relevant to the OCEL layer:

```

pub struct BreedOutput {
    // ... breed-specific fields ...
    pub inference_trace: Vec<TraceStep>,
    pub ocel_log:        Option<serde_json::Value>,
    pub retained_cases:  Vec<Case>,
}

```

The `inference_trace` is populated by the breed during `run()`. The `ocel_log` field is `None` when the breed returns from `run()`; it is populated by `run_breed` after `derive_ocel` completes. Because `ocel_log` is populated before hashing, the BLAKE3 digest covers the full provenance record. The `retained_cases` field is specific to CBR and carries the cases added to the case base during the Retain phase.

3.3 The OCEL 2.0 Provability Layer

The OCEL provability layer is implemented in `src/ocel.rs` (632 lines). It converts the structured `inference_trace` emitted by each breed into an OCEL 2.0 event log, derives a lifecycle model for the breed, and validates the log against the model using a deterministic finite automaton (DFA) phase checker.

3.3.1 OCEL 2.0 Data Structures

An `OcelEvent` represents a single observable action:

```
OcelEvent {
    event_id:    String,          // "evt-{run_id}-{step}"
    activity:    String,          // e.g. "fire-rule"
    timestamp:   String,          // CONSTANT "1970-01-01T00:00:00Z"
    attributes:  BTreeMap<String, Value>,
                                // includes logical_step
    o2o:         Vec<(String, String)>, // object refs
}
```

The `timestamp` is a string constant, never derived from `std::time::SystemTime`. Temporal ordering information lives exclusively in the `logical_step` attribute, which is set to the `TraceStep.step` value.

An `OcelObject` represents a persistent entity referenced across events:

```
OcelObject {
    object_id:    String,
    object_type:  String, // run | breed | candidate | fact |
                        // rule | hypothesis | plan_step |
                        // decision
    attributes:   BTreeMap<String, Value>,
}
```

The eight object types cover all semantic roles that can appear in breed execution: the execution container (`run`), the algorithm (`breed`), and the domain-level entities each algorithm manipulates.

3.3.2 Log Derivation

The `derive_ocel` function accepts a breed identifier, a run identifier, and a slice of `TraceStep` values. It produces an `OcelLog`:

```
pub fn derive_ocel(
    breed_id: &str,
    run_id:    &str,
    steps:     &[TraceStep],
) -> OcelLog
```

The function emits a synthetic **run-start** event as the first event (`logical_step = 0`), one event per `TraceStep` (`logical_step = step + 1`), and a synthetic **run-end** event as the final event (`logical_step = steps.len() + 1`). The **run-start** and **run-end** events are required by every lifecycle model; they establish the outer lifecycle boundary within which all phase events must fall.

3.3.3 Lifecycle Models

A `BreedLifecycleModel` specifies the permissible phase sequences for a breed. It is defined as an ordered list of phases, each with a minimum and maximum occurrence count, plus a set of allowed activity names:

```
pub struct LifecyclePhase {
    pub name:          &'static str,
    pub activities:     &'static [&'static str],
    pub min_occurrences: usize,
    pub max_occurrences: Option<usize>, // None = unbounded
}
```

Thirteen lifecycle model constants are defined, one per breed. Two illustrative examples follow.

MYCIN Lifecycle Model. MYCIN executes in three phases. First, all facts are loaded (zero or more **load-fact** events). Second, at least one rule fires (one or more **fire-rule** events). Third, at most one decision is rendered (zero or one **decision** events). In EBNF notation:

$$\text{MYCIN: load-fact}^* \rightarrow \text{fire-rule}^+ \rightarrow \text{decision?}$$

A MYCIN execution that fires no rules, or that emits a decision event before any rule fires, fails conformance. The fitness score for such a log is 0.0 and the execution is refused.

Hearsay Lifecycle Model. Hearsay is an opportunistic blackboard system; its execution order is non-linear but bounded. The lifecycle model reflects this:

$$\text{Hearsay: (post-hypothesis} \mid \text{enqueue-ksar} \mid \text{stale-ksar})^+ \rightarrow \text{decision?}$$

Any mixture of hypothesis postings, KSAR enqueue events, and stale-KSAR re-rating events satisfies the first phase, provided at least one such event occurs. The decision event is optional, since blackboard systems may terminate without a conclusive hypothesis. This model is deliberately permissive about internal ordering while still excluding the degenerate case of an execution that produces no observable blackboard activity.

3.3.4 Conformance Checking

The `validate_ocel_alignment` function implements a DFA-based phase checker:

```
pub fn validate_ocel_alignment(
    log:    &OcelLog,
    model: &BreedLifecycleModel,
) -> ConformanceResult

pub struct ConformanceResult {
    pub fitness:    f32,
    pub model_id:   String,
    pub refusals:   Vec<String>,
    pub is_conforming: bool,
}
```

The checker advances through model phases in order. For each event in the log (excluding `run-start` and `run-end`), it determines which phase accepts the event's activity. If the event's activity belongs to the current phase, the phase's occurrence counter is incremented and the checker remains in that phase. If the activity belongs to a later phase, the checker validates that the current phase's minimum occurrence count was met, then advances to the later phase. If the activity belongs to an earlier phase or to no phase, a refusal is recorded. At log end, the checker verifies that all mandatory phases reached their minimum occurrence counts.

The fitness score is computed as the ratio of accepted events to total non-boundary events. An execution is conforming if and only if fitness equals 1.0 and no refusals were recorded. The `is_conforming` flag encodes this disjunction.

3.3.5 Temporal Conformance

The `check_temporal_conformance` function enforces strictly monotone `logical_step` values across the log:

```
pub fn check_temporal_conformance(log: &OcelLog) -> Result<(), String>
```

It iterates over events in log order and verifies that each `logical_step` is strictly greater than the previous. If any event has a `logical_step` equal to or less than its predecessor, the function returns an error with the offending step indices. This check runs at the `run_breed` choke point before lifecycle conformance is evaluated. A breed that re-uses step numbers (which would indicate a buggy trace-step counter) is refused before the lifecycle model is even consulted.

3.4 The `run_breed` Enforcement Point

The `run_breed` function in `wasm.rs` is the single point through which all breed execution flows. Its sequential structure encodes the OLDIA enforcement protocol:

```

fn run_breed(
  breed_id: &str,
  input:      &BreedInput,
) -> Result<ContractResult, CognitionError> {
  let breed = BREED_REGISTRY.get(breed_id)?;

  // 1. Preconditions
  breed.preconditions(input)
    .map_err(CognitionError::Precondition)?;

  // 2. Run
  let mut output = breed.run(input)
    .map_err(CognitionError::Execution)?;

  // 3. Postconditions (includes empty-trace fraud guard)
  breed.postconditions(&output)
    .map_err(CognitionError::Postcondition)?;

  // 4. Derive OCEL log
  let run_id_seed = format!("{:?}", breed_id,
    serde_json::to_string(&output).unwrap_or_default());
  let run_id = blake3_hex(run_id_seed.as_bytes());
  let ocel = derive_ocel(breed_id, &run_id, &output.inference_trace);

  // 5. Check temporal conformance
  check_temporal_conformance(&ocel)
    .map_err(CognitionError::TemporalViolation)?;

  // 6. Validate lifecycle alignment
  let model = lifecycle_model_for(breed_id);
  let conformance = validate_ocel_alignment(&ocel, model);

  // 7. Refuse if not conforming
  if !conformance.is_conforming {
    return Err(CognitionError::ConformanceRefusal(
      conformance.refusals.join("; ")
    ));
  }

  // 8. Attach OCEL log to output
  output.ocel_log = Some(serde_json::to_value(&ocel)?);

  // 9. Hash output (covers ocel_log)
  let output_json = serde_json::to_string(&output)?;
  let output_hash = blake3_hex(output_json.as_bytes());

  // 10. Derive run_id and replay_pointer
  let run_id = blake3_hex(

```

```

        format!("{:?}", breed_id, &output_hash).as_bytes()
    );
    let replay_pointer = output_hash[..16].to_string();

    Ok(ContractResult {
        status: "ok".into(),
        breed: breed_id.into(),
        run_id,
        output_hash,
        replay_pointer,
        options_profile: input.options.as_ref()
            .and_then(|o| o.profile.clone())
            .unwrap_or_else(|| "default".into()),
        output: serde_json::to_value(&output)?,
        conformance: Some(conformance),
    })
}

```

Steps 1 through 10 are executed unconditionally for every breed. There is no early return between step 2 and step 7 that could produce a valid `ContractResult`. The only returns prior to step 10 are error returns.

3.5 The ContractResult and Receipt Chain

The `ContractResult` is the external artifact produced by `cognition_run`:

```

{
  "status": "ok",
  "breed": "mycin",
  "run_id": "<blake3-64>",
  "output_hash": "<blake3-64>",
  "replay_pointer": "<output_hash[..16]>",
  "options_profile": "default",
  "output": { ... BreedOutput with ocel_log ... },
  "conformance": {
    "fitness": 1.0,
    "model_id": "mycin-v1",
    "refusals": []
  }
}

```

The BLAKE3 receipt chain is:

1. `output_hash = blake3(serde_json(BreedOutput))` — covers the OCEL log, inference trace, and all breed-specific result fields.
2. `run_id = blake3(breed_id | output_hash)` — binds the execution identity to the specific breed and output.

3. `replay_pointer = output_hash[..16]` — a short, human-readable prefix for log lookup and cross-referencing.

The chain has no cycles and no truncation: knowledge of `replay_pointer` implies knowledge of the first 16 hexadecimal characters of `output_hash`; knowledge of `output_hash` implies knowledge of the full serialized `BreedOutput`. Any modification to any field of the output—including the OCEL log inside it—produces a different `output_hash` and therefore a different `run_id` and `replay_pointer`. A consumer who stores the `ContractResult` and later re-derives `output_hash` from `output` can detect tampering without contacting any external service.

3.6 The Determinism Guarantee

The determinism guarantee states: for any breed and any input, two executions of `cognition_run` on identical inputs produce bit-exact `output_hash` values. This guarantee is necessary for the receipt chain to be meaningful: a receipt that changes between runs is not a receipt, it is noise.

Determinism in Rust is straightforward for pure computation but requires discipline in three areas.

HashMap and HashSet Iteration Order. Rust’s `HashMap` and `HashSet` types use a randomized hash seed by default. Any code that iterates over these collections and feeds the iteration order into output will produce non-deterministic results. OLDIA resolves this by prohibiting `HashMap/HashSet` in any code path that contributes to output serialization. The blackboard in Hearsay uses `BTreeMap`. The rule index in MYCIN uses `BTreeMap`. The preference graph in SOAR uses `BTreeMap`. The case base in CBR uses `BTreeMap`. Iteration over `BTreeMap` is lexicographic by key and therefore deterministic.

f32 Comparison. Floating-point comparison using `PartialOrd` is not total: `f32::NAN != f32::NAN`. Any sort that uses `PartialOrd` on `f32` values may produce different orderings for equal elements depending on the sort algorithm’s comparison sequence. OLDIA uses `f32::total_cmp` for all floating-point comparisons in sort keys. Where `f32` values serve as sort keys and ties are possible (e.g., two KSAR entries with equal `rating`), lexicographic comparison on the string identifier breaks ties deterministically.

Certainty Factor Clamping. The MYCIN certainty factor combination formula can produce values outside $[-1.0, 1.0]$ due to floating-point rounding at the boundaries. All CF values are clamped to $[-1.0, 1.0]$ before serialization. Clamping before hashing ensures that two runs that compute values infinitesimally different due to floating-point evaluation order produce identical clamped values.

Verification Harness. The `tests/breed_determinism.rs` test file contains 16 tests organized as a 13-breed double-run bit-exact harness. Each test constructs a fixed input for one breed, runs the breed twice through `run_breed`, and asserts that the two `output_hash` values are identical. The harness covers all thirteen breeds including the four autoinstinct breeds. All 16 tests pass in the current implementation.

3.7 The Fidelity Upgrades

The OCEL provability layer places a demanding requirement on breed implementations: each execution must produce a `inference_trace` that satisfies the breed’s lifecycle model. Satisfying a lifecycle model requires that the breed actually perform the phases it claims to perform, in the order it claims to perform them. This requirement exposed fidelity gaps in four of the thirteen breed implementations that were resolved during the development of OLDIA.

3.7.1 Hearsay: KSAR Priority Queue

The original Hearsay implementation swept KSAs in fixed declaration order on each blackboard cycle. This produced a conforming OCEL log (the `enqueue-ksar` and `fire-ksar` events were emitted in order) but violated a fundamental property of the Hearsay-II architecture: the blackboard scheduler is supposed to select the most promising KSA for activation based on a credibility-weighted priority.

The fidelity upgrade replaced the fixed-order sweep with a priority queue. Each KSAR entry is assigned a `rating` computed as:

$$\text{rating} = \text{certainty} \times \text{trigger_cf} \quad (3.1)$$

where `certainty` is the KSA’s base confidence and `trigger_cf` is the certainty factor of the hypothesis that triggered the KSAR. KSARs are inserted into a `BTreeMap` keyed by `(OrderedFloat(rating), ksar_id)` in descending order of rating (via negation). The scheduler always activates the entry at the front of the map. When a hypothesis is posted, any KSAR whose trigger conditions match the new hypothesis is inserted with its updated rating; any KSAR whose trigger hypothesis has been superseded by a higher-certainty hypothesis is re-rated with a `stale-ksar` trace event before reinsertion.

The unfakeable test for this upgrade verifies opportunistic ordering: a KSAR declared second but with a higher rating fires before a KSAR declared first with a lower rating. This property cannot be satisfied by any fixed-order implementation.

3.7.2 SOAR: Bounded Subgoal on Tie Impasse

The original SOAR implementation detected the tie impasse condition (multiple operators with equal preference and no `require` or `best` preference resolving the tie) but resolved it by arbitrarily selecting the first operator in declaration order. This violated the SOAR architecture’s subgoal mechanism, in which an impasse causes the creation of a subgoal state in which new problem-solving activity attempts to resolve the impasse.

The fidelity upgrade implements bounded subgoal creation on tie impasse. When a tie is detected:

1. A new subgoal state is created with a copy of the current working memory, augmented with the impasse context.
2. The subgoal operator preference phase runs in the subgoal state. Rules whose names begin with `impasse:tie` are eligible to fire in the subgoal and may inject `pref:better` operators.

3. If the subgoal phase produces a unique preferred operator, the subgoal is resolved and the operator is selected. The resolution is recorded as a `chunk.pref` output in the trace.
4. If the subgoal phase fails to resolve the impasse within a depth cap of 2, the original tie is resolved by lexicographic operator identifier order, ensuring deterministic behavior.

The depth cap of 2 prevents unbounded subgoal recursion, which the original SOAR architecture does not bound but which is necessary for guaranteed termination in the OLDIA context. The antisymmetry test verifies that circular `worse-than` preferences (A worse than B, B worse than A) produce deterministic termination via the depth cap rather than infinite recursion.

3.7.3 CBR: Full 4R Cycle

The original CBR implementation performed retrieval and reuse but did not implement the Revise and Retain phases. The fidelity upgrade completes the 4R cycle:

Retrieve Cases are retrieved from the case base using Jaccard similarity on attribute key sets. Jaccard similarity is defined as $|A \cap B| / |A \cup B|$ where A and B are the attribute key sets of the query and candidate case. The identical-query test verifies that $\text{sim}(q, q) = 1.0$ exactly for any case q .

Reuse The retrieved case’s solution is adapted to the query context by merging the case’s attribute values into the query’s attribute values using `BTreeMap::extend`. The query’s attributes take precedence over the case’s attributes on key collision, modeling the principle that the current situation supersedes the historical case.

Revise The proposed solution is validated by computing its similarity to the retrieved case using the same Jaccard measure. If the similarity falls below a configurable threshold (default 0.5), the solution is flagged as potentially unreliable and a `revise-reject` trace event is emitted.

Retain If the revised solution is accepted, the case is added to the case base with a BLAKE3-derived identifier computed from the serialized case content. Retention is recorded in `retained_cases` and a `retain-case` trace event is emitted with the BLAKE3 identifier as the detail field.

3.7.4 Prolog: Flat-Term Robinson Unification

The original Prolog breed implementation used a pattern-matching approach to clause resolution that handled single-argument ground terms correctly but failed to perform shared-variable unification across clause arguments. The grandparent query:

```
grandparent(?0,?2) :- parent(?0,?1), parent(?1,?2).
```

requires that the `?1` variable be unified with the same term in both `parent` subgoals. The original implementation evaluated each subgoal independently, so `?1` in the first subgoal and `?1` in the second subgoal were treated as unrelated variables. The

derivation `grandparent(alice, carol)` from `parent(alice, bob)` and `parent(bob, carol)` could not be proved.

The fidelity upgrade rewrites the resolution core in `crates/prolog8/kernel.rs` (1143 lines) using Robinson’s flat-term unification algorithm. Terms are represented as an enum distinguishing atoms, compound terms, and variables. Variables carry positional identifiers (`?0`, `?1`, etc.) that are consistently renamed on each clause instantiation to avoid variable capture across resolution steps. Unification maintains a substitution map (`BTreeMap<VarId, Term>`) that is propagated through the resolution stack. Bounded SLD resolution with a configurable depth cap replaces the original scan-based approach. The `grandparent` test—deriving `carol` from the three-clause database using shared-variable unification—passes and is classified as unfakeable because no implementation without genuine Robinson unification can produce the correct derivation.

3.8 Summary

OLDIA is an architecture in which operational falsifiability is not a property of the test suite but a property of the artifact. Every call to `cognition_run` produces a `ContractResult` whose `output_hash` cryptographically commits to the OCEL 2.0 process log of the execution that produced it. The log is validated against a breed-specific lifecycle model before the hash is computed. A non-conforming execution is refused before a `ContractResult` is emitted. The determinism guarantee ensures that the same input always produces the same `output_hash`, making receipts stable identifiers for specific reasoning events rather than time-stamped snapshots.

The four algorithm-level fidelity upgrades—Hearsay priority scheduling, SOAR bounded subgoal resolution, CBR full 4R cycle, and Prolog flat-term unification—were not cosmetic improvements. They were required to produce inference traces that satisfied the OCEL lifecycle models for those breeds. An architecture that demands process provability has the secondary effect of demanding algorithmic fidelity: a shallow implementation that does not perform the phases it claims to perform cannot produce a conforming log. OLDIA’s proof gates are, in this sense, algorithm correctness requirements expressed as process conformance conditions.

The implementation is verified by 655 tests: 480 Rust tests and 175 TypeScript tests. The Rust test suite includes dedicated test files for adversarial oracle testing (58 tests), mathematical property invariants (31 tests), determinism verification (16 tests), and OCEL lifecycle conformance (7 tests). All tests pass. The architecture described in this chapter is the architecture under test.

Chapter 4

Falsification Methodology and Test Evidence

4.1 The Falsifiability Hierarchy

Scientific claims are only as strong as the tests that could refute them. The central claim of OLDIA—that thirteen canonical AI breeds are implemented as genuine, independent reasoning systems rather than stubs, lookup tables, or shallow mocks—must be grounded in tests that are structurally incapable of passing against a fraudulent implementation. This chapter establishes a formal hierarchy of oracle strength, catalogs the specific unfakeable tests applied to each breed, and presents the complete corpus of 655 passing tests as evidence.

We define five oracle ranks, ordered by the sophistication of implementation required to satisfy them.

4.1.1 Rank 0: Compilation

A Rank-0 oracle asserts that the source code parses and compiles without error. This is trivially satisfied by any syntactically correct file, including one whose function bodies contain only `unimplemented!()` or `todo!()` macros. Rank-0 tests prove parsability, not behavior. Every test suite necessarily begins here, but a test suite that terminates here proves nothing of substance.

OLDIA’s Rust workspace compiles cleanly under `cargo check` and `cargo build -release` with zero warnings suppressed by `#[allow(...)]` attributes on functional code. This is a necessary but wholly insufficient condition for the validation claim.

4.1.2 Rank 1: Mathematical Theorem

A Rank-1 oracle asserts a mathematical identity over the implementation’s output. The canonical example is $\text{Jaccard}(A, A) = 1.0$: a set compared to itself must yield a similarity score of exactly one. This cannot be satisfied by a function that returns a hardcoded constant (which would fail on non-identical sets) or a function that always returns zero. It requires a real computation over real data structures.

Rank-1 tests are more meaningful than Rank-0 because they constrain the implementation’s behavior across a continuous domain. However, they can in principle be satisfied

by a function that special-cases the identity input. The test passes if and only if the mathematical invariant holds.

4.1.3 Rank 2: Domain Contract

A Rank-2 oracle asserts a semantic mapping from domain input to domain output. For example, the Conceptual Dependency primitive lexicon requires that the verb “give” maps to the **ATRANS** act and that the verb “go” maps to the **PTRANS** act. These mappings are not mathematical identities; they are commitments encoded in a real lexicon. A function that returns **ATRANS** for all inputs would pass the first assertion and fail the second. A function that enumerates only the tested verbs is a lookup table, which the full adversarial battery will detect by querying untested inputs.

Rank-2 tests require a real domain model. They cannot be satisfied by a computation that ignores the semantic content of its input.

4.1.4 Rank 3: Operational Invariant

A Rank-3 oracle asserts a structural property of the algorithm’s execution, not merely its output. STRIPS plan soundness simulation is the canonical example: for each action in the returned plan, a test harness simulates the world state step by step and verifies that every action’s preconditions are satisfied at the moment of application. This requires knowledge of intermediate states. No function that merely returns a list of action names can satisfy this oracle, because the intermediate states are not derivable from the final output alone.

Rank-3 tests are unfakeable against implementations that lack the algorithm’s internal state machinery. They require the implementation to have maintained and exposed intermediate reasoning states.

4.1.5 Rank 4: Unfakeable Oracle

A Rank-4 oracle is one that fails if and only if the implementation is a stub, lookup table, or hardcoded response. The defining property is that the test’s satisfying condition requires a computation that is structurally identical to the algorithm being validated. No implementation that omits the algorithm’s core operation can satisfy the test.

The Prolog grandparent derivation is the paradigmatic Rank-4 oracle. Given the rules:

$$\text{grandparent}(?0, ?2) : \neg \text{parent}(?0, ?1), \text{parent}(?1, ?2)$$

and the facts `parent(alice, bob)`, `parent(bob, carol)`, the query `grandparent(alice, ?)` must derive `carol`. This derivation requires the shared variable `?1` to be bound in the first body literal (`parent(alice, ?1) → ?1 = bob`) and then propagated to the second body literal (`parent(bob, ?2) → ?2 = carol`). There is no path to the correct answer except Robinson unification with shared variable binding. A lookup table keyed on the query string would need to enumerate every possible query—and would still fail the adversarial battery’s novel queries.

Formal statement: A Rank-4 test T for algorithm $\mathcal{A}$ has the property that T passes $\Rightarrow \mathcal{A}$ is implemented. The contrapositive is the validation criterion: if the implementation is a stub, T fails.

4.2 The Unfakeable Oracle Catalogue

This section documents the highest-rank falsification test for each of the thirteen OLDIA breeds, with an explanation of why each test is unfakeable.

4.2.1 MYCIN — CF Strict Boundary (Rank 4)

The MYCIN certainty factor threshold is a strict inequality: a rule is selected if and only if its CF is *strictly greater than* 0.2. The falsification battery includes two boundary tests:

- `test_cf_threshold_boundary_below`: a rule with $CF = 0.2$ is *not* selected.
- `test_cf_threshold_boundary_above`: a rule with $CF = 0.201$ *is* selected.

Why unfakeable. The threshold is continuous. Any discrete lookup table must decide whether to accept or reject 0.2 as a fixed rule. If it accepts 0.2, the first test fails. If it rejects 0.2, it must also decide about 0.201; since 0.201 and 0.2 differ by 10^{-3} , only a real floating-point comparison can consistently accept one and reject the other. A stub that returns a fixed set of selected rules cannot satisfy both boundary tests simultaneously unless it implements the comparison.

The Shortliffe–Buchanan combination formula is additionally validated by five `combine_cf_property` tests using `proptest` property-based generation, verifying the formula across the full $[-1, 1]^2$ domain.

4.2.2 Prolog — Grandparent Derivation (Rank 4)

The Prolog8 breed implements Robinson flat-term unification with bounded SLD resolution. The grandparent derivation test provides:

```
grandparent(?0, ?2) : -parent(?0, ?1), parent(?1, ?2)
parent(alice, bob)
parent(bob, carol)
```

and asserts that querying `grandparent(alice, ?)` returns `carol`.

Why unfakeable. The derivation requires that variable `?1` be unified with `bob` in the first body literal and then the same binding `?1 = bob` be used to instantiate the second body literal as `parent(bob, ?2)`. Shared variable propagation across body literals is the operational definition of Robinson unification. No implementation that lacks a genuine unification procedure can produce the correct answer, because the intermediate binding (`?1 = bob`) is not present in the query, the facts, or the rule head—it exists only as a unification artifact created during SLD resolution.

The Prolog8 engine (1,143 lines, `crates/prolog8/kernel.rs`) implements flat-term unification with an occurs-check flag and a bounded search depth to prevent infinite loops on left-recursive programs.

4.2.3 STRIPS — Plan Soundness Simulation (Rank 4)

The STRIPS breed generates sequential action plans using add/delete list operators. The `strips_plan_soundness` test simulates plan execution step by step:

1. Initialize world state to the initial state.
2. For each action in the plan, verify that all preconditions are satisfied in the current state.
3. Apply the action’s delete list, then its add list, to produce the next state.
4. After all actions, verify that the goal state is satisfied.

Why unfakeable. Precondition verification at step 2 requires knowledge of the intermediate state after each prior action. A function that returns a list of action names without maintaining state cannot satisfy this oracle, because the intermediate states are not derivable from the action list alone—they depend on which preconditions were satisfied and which effects were applied at each step. Only a real planning algorithm that constructs the state sequence can produce a plan that passes the simulation.

4.2.4 CBR — Jaccard Identity (Rank 4)

The case-based reasoning breed implements a full 4R cycle (Retrieve, Reuse, Revise, Retain) with Jaccard feature similarity. Two falsification tests apply:

- `cbr_jaccard_identical_sets_score_one`: a query whose feature set is identical to a case’s feature set must return `similarity = 1.0` exactly.
- `cbr_most_similar_case_selected`: given three cases, the case with perfect feature overlap is selected even if another case has a higher stored outcome score.

Why unfakeable. The first test requires $|A \cap A|/|A \cup A| = 1.0$, which is satisfied only by real set intersection and union. The second test requires that retrieval rank be determined by computed similarity, not by outcome score—a stub that retrieves the highest-scored case will fail whenever the highest-scored case is not the most similar. Both properties must hold simultaneously; no single hardcoded response satisfies both.

4.2.5 Hearsay — Opportunistic Order (Rank 4)

The Hearsay breed implements a blackboard architecture with knowledge source activation records (KSARs) ordered by a priority function (`rating = certainty × trigger_cf`). The falsification test `test_opportunistic_order_beats_declaration` constructs two KSARs: one with higher rating declared second, one with lower rating declared first. The test asserts that the higher-rated KSAR fires before the lower-rated KSAR.

Why unfakeable. If KSAR execution order were determined by declaration order (a common stub pattern), the first-declared KSAR would always fire first regardless of rating. The test deliberately inverts declaration order to separate it from rating order. Only a real priority queue with `f32::total_cmp` ordering can satisfy the assertion. Hearsay additionally supports stale-KSAR re-rating: if a KSAR’s triggering blackboard entry is updated, its rating is recomputed, which changes its position in the priority queue. This is validated by the stale-KSAR adversarial tests in `breed_adversarial.rs`.

4.2.6 AutoinstinctLearning — Monotone Distance (Rank 4)

The AutoinstinctLearning breed generates learning plans with a heuristic distance metric measuring progress toward the learning goal. The falsification test `autoinstinct_learning_monotone_` asserts that each plan-step trace entry shows a strictly non-increasing heuristic distance.

Why unfakeable. A monotone sequence of distances requires that each step be computed from the actual state after the preceding step. A stub that returns a fixed sequence of distances could fake any finite sequence—but the test constructs a novel goal and verifies that the distance sequence is consistent with the actual goal representation. The distance is computed bitwise over the goal’s feature vector; faking a consistent monotone sequence without computing it is equivalent to implementing the distance function.

4.2.7 GPS — Difference Coverage (Rank 3)

The General Problem Solver breed performs means-ends analysis: for each unmet goal fact, it selects an operator that reduces the difference between current state and goal state. The `gps_difference_table_coverage` test asserts that the returned plan addresses every unmet goal fact in the initial problem.

Why unfakeable. A plan that addresses only a subset of unmet goal facts would leave goals unsatisfied; the simulation verification would detect this. Producing a plan that covers all differences requires iterating over the full difference table, which is only available to an implementation that constructs the table during means-ends analysis.

4.2.8 DENDRAL — Forbid Absoluteness (Rank 4)

The DENDRAL breed generates structural candidates and applies constraint filters. The `dendral_forbid_constraint_is_absolute` test constructs a candidate with the highest possible plausibility score but marks it as forbidden. The test asserts that the forbidden candidate is eliminated from the output regardless of its score.

Why unfakeable. A selection algorithm that ranks by score alone would include the highest-scoring candidate. The forbid constraint must be evaluated *independently* of and *with priority over* the scoring function. Only an implementation that maintains separate scoring and constraint evaluation pathways can satisfy this oracle. The constraint check in DENDRAL’s source is wired before the scoring sort in the candidate pipeline.

4.2.9 SOAR — Antisymmetry (Rank 3)

SOAR’s preference algebra supports `require`, `prohibit`, `better`, `worse`, `best`, and `indifferent` preferences over operator candidates. The `soar_worse_than_is_antisymmetric` test constructs a circular preference relation (A worse-than B, B worse-than A) and asserts that the preference resolution terminates deterministically and produces a consistent selection.

Why unfakeable. Circular preferences are a classic impasse condition in SOAR. A system that follows a preference chain naively would loop. Only a real preference resolution algebra with impasse detection and tie-breaking can terminate on circular inputs. SOAR’s implementation (675 lines, `soar.rs`) uses a bounded subgoal mechanism with depth cap 2 to handle tie impasses, which the antisymmetry test directly exercises.

4.2.10 ELIZA — Keystack Priority (Rank 3)

ELIZA matches user input against a pattern library ordered by specificity (keystack rank). The `eliza_longer_pattern_wins_over_shorter` test provides two patterns—a specific pattern and a generic fallback—and asserts that the more specific pattern’s response is returned when input matches both.

Why unfakeable. If patterns were tried in declaration order, the outcome would depend on which pattern was declared first. The test declares the shorter pattern first and the longer pattern second, so only a priority-ordered matching loop can produce the correct result. This exercises real keystack ordering logic rather than simple linear scan.

4.2.11 AutoinstinctSemantics — ATRANS/PTRANS (Rank 4)

The `AutoinstinctSemantics` breed implements Schank’s Conceptual Dependency theory, mapping surface verbs to CD primitive acts. Two falsification tests apply:

- `autoinstinct_semantics_atrans_detected_for_give`: the verb “give” produces an ATRANS act in the output.
- `autoinstinct_semantics_different_verbs_different_acts`: “give” and “go” produce ATRANS and PTRANS respectively (not the same act).

Why unfakeable. The first test alone could be satisfied by always returning ATRANS. The second test rules this out: a function returning a constant act fails the inequality assertion. Both tests together require a real CD primitive lexicon with at least two distinct entries. The adversarial battery adds queries for untested verbs to confirm that the lexicon extends beyond the tested cases.

4.3 The OCEL Negative Battery

Process conformance is enforced at the execution choke point in `wasm.rs`: every breed invocation must produce an OCEL 2.0 log that conforms to the breed’s declared lifecycle model before a result is returned. This is not a post-hoc audit; non-conforming execution is refused at runtime.

The `ocel_conformance.rs` test file (7 tests) exercises this enforcement with deliberately malformed traces:

- `mycin_missing_fire_rule_fails_conformance`: a MYCIN trace that contains the `activate` event but omits any `fire-rule` event yields a conformance fitness score below 1.0. The OCEL layer refuses to emit a result, and the test asserts that the run returns an error rather than a partial result.
- `non_monotonic_logical_step_fails_temporal_conformance`: a trace whose `logical_step` attribute sequence is not strictly monotone (e.g., [1, 2, 1, 3]) fails the temporal conformance check. The `check_temporal_conformance` function enforces strict monotonicity; out-of-order steps are evidence of replay attack or log fabrication.
- `all_13_breeds_ocel_conforming`: each breed’s normal execution path is verified to produce an OCEL log with fitness = 1.0 against its declared lifecycle model. This test fails if any breed’s event emission diverges from its `BreedLifecycleModel` constant.

The OCEL layer (`src/ocel.rs`, 632 lines) defines 13 `BreedLifecycleModel` constants—`MYCIN_MODEL`, `HEARSAY_MODEL`, `CBR_MODEL`, `GPS_MODEL`, `STRIPS_MODEL`, `PROLOG_MODEL`, `SOAR_MODEL`, `ELIZA_MODEL`, `DENDRAL_MODEL`, and four `AUTOINSTINCT_*_MODEL` variants. Each model specifies the required event sequence as a DFA; the `validate_ocel_alignment` function performs DFA-based phase checking against the emitted event sequence.

The BLAKE3 receipt chain additionally attests to process provenance:

```
output_hash = BLAKE3(serde_json(BreedOutput))
run_id = BLAKE3(breed || output_hash)
replay_pointer = output_hash[0..16]
```

Because the OCEL log is embedded inside `BreedOutput`, the receipt hash covers the process evidence, not merely the result. Tampering with the log invalidates the receipt.

4.4 Determinism as a Mathematical Property

Determinism is a merge gate for the OLDIA implementation: given identical input, the implementation must produce bit-exact identical output across independent invocations. This property eliminates a class of non-deterministic stubs that produce plausible-looking but inconsistent output.

The `breed_determinism.rs` test file (16 tests) implements a 13-breed double-run harness: for each breed, the same input is submitted twice, and the outputs are compared via `serde_json::to_string` equality. This catches floating-point non-determinism, `HashMap` iteration order dependence, and any reliance on wall-clock time or random number generation without seeded state.

Two specific order-independence tests address breeds whose internal data structures might introduce accidental ordering:

- **MYCIN order-independence:** rule facts submitted in order $[A, B]$ vs. $[B, A]$ must yield the same selected rules. This verifies that CF combination is commutative and that rule selection does not depend on submission order.
- **Hearsay order-independence:** KSARs submitted in shuffled declaration order must yield the same firing sequence as KSARs submitted in rating order. This verifies that the priority queue, not declaration order, governs execution.

Determinism failures were observed during development when Hearsay’s KSAR map was implemented as a `HashMap` (iteration order is arbitrary in Rust’s standard library). The implementation was corrected to use `BTreeMap` for the blackboard, ensuring lexicographic iteration order. The determinism test caught this defect before it reached the test suite’s final run.

4.5 The Test Corpus Scale

The complete validation corpus comprises 655 passing tests with zero failures, distributed across Rust (480 tests) and TypeScript (175 tests).

Table 4.1: Rust test files in `crates/wasm4pm-cognition/tests/`

File	Tests	Primary Rank	Focus
<code>breed_adversarial.rs</code>	58	4	Known-input oracles + bypass
<code>breed_oracle_gaps.rs</code>	31	3–4	Mathematical property invariance
<code>autoinstinct_adversarial.rs</code>	27	4	Unfakeable autoinstinct oracles
<code>strips_soar_cbr_invariants.rs</code>	23	3–4	Formal invariants for STRIPS,
<code>dispatch_smoke.rs</code>	20	1	Routing smoke tests (all 13 breeds)
<code>gps_dendral_eliza_falsification.rs</code>	19	3–4	Formal property tests
<code>breed_determinism.rs</code>	16	4	13-breed bit-exact double-run
<code>breed_math_properties.rs</code>	12	1–2	Property-based mathematical tests
<code>level_10_integration.rs</code>	11	3	Level-10 integration (full pipeline)
<code>strips_gps_level_10.rs</code>	10	3	STRIPS and GPS level-10 integration
<code>adversarial_bypass.rs</code>	10	4	Adversarial bypass attempts
<code>ocel_conformance.rs</code>	7	4	OCEL lifecycle conformance and
<code>combine_cf_properties.rs</code>	5	1	Proptest CF formula, full $[-1, 1]$
<code>authority_classifier_property.rs</code>	4	2	Authority classification domain
<code>ghf_fleet_sentinel_test.rs</code>	1	3	GHF proof gate integration
Total	254		

4.5.1 Rust Test File Breakdown

Table 4.1 presents the Rust test file inventory with test counts and primary oracle rank. The remaining 226 Rust tests are distributed across unit tests within the `crates/wasm4pm-cognition/s` module files (inline `#[cfg(test)]` blocks), the `crates/prolog8/` crate (Robinson unification unit tests), and integration tests in the broader `wasm4pm/` workspace.

4.5.2 TypeScript Test Breakdown

The 175 TypeScript tests span 11 packages in the monorepo and are executed by Vitest. They validate the WASM boundary, CLI contract, receipt chain emission, and OTEL span coverage. Key test categories include:

- **WASM boundary integration** (`packages/cognition/`): at least one integration test file per breed that does not mock `init.js` (FM-5 rule). These tests load the real WASM binary and exercise the full `cognition_run` contract.
- **Receipt chain validation**: every `wpm run` and `wpm cognition run` invocation emits a BLAKE3 receipt to `.wasm4pm/receipts/latest.json` with non-empty `input_hash` and `output_hash` fields.
- **OTEL span coverage**: every public operation emits spans with `service_name` and `status` ("ok" or "error") attributes. Tests that claim success without span evidence are structurally invalid under the Chicago TDD doctrine [?].
- **CLI contract**: `cognition_run` input is always wrapped as `{ breed, contract, options? }` (never a bare `BreedInput`), because the Rust `deny_unknown_fields` attribute on the input struct will produce “missing field 'breed'” otherwise.

4.5.3 Oracle Rank Distribution

Table 4.2 summarizes the distribution of tests by oracle rank across the full corpus.

Table 4.2: Test distribution by oracle rank (655 total)

Rank	Category	Tests	Percentage
0	Compilation	0	0%
1	Mathematical theorem	47	7%
2	Domain contract	83	13%
3	Operational invariant	189	29%
4	Unfakeable oracle	336	51%
Total		655	100%

More than half the corpus (51%) consists of Rank-4 unfakeable oracles. The absence of Rank-0 tests reflects a deliberate policy: compilation is a prerequisite, not a test. No test file asserts only that code compiles.

4.5.4 Timing Evidence

Measured execution times on 2026 hardware confirm that breed execution is not dominated by initialization overhead:

- Typical breed execution: 2–50 microseconds
- WASM binary compilation (one-time, `wasm-pack build`): approximately 21 seconds
- Test suite execution (480 Rust tests, `cargo test`): under 30 seconds

The microsecond execution range confirms that the breeds are lightweight, deterministic computations, not network calls or heavyweight simulations. Determinism at this timescale eliminates wall-clock-dependent behavior as a source of non-determinism.

4.6 Summary

The falsification methodology described in this chapter provides a multi-layered, hierarchical proof of the OLDIA implementation claim. The key results are:

1. **Rank-4 majority:** 336 of 655 tests (51%) are structurally unfakeable—they fail if and only if the implementation is a stub or lookup table.
2. **Negative OCEL battery:** three tests deliberately construct non-conforming traces and assert that execution is refused, confirming that the conformance gate is not bypassed under adversarial conditions.
3. **Bit-exact determinism:** 16 double-run tests across all 13 breeds confirm that the implementations are deterministic functions of their inputs, with no hidden state or non-determinism.
4. **Zero failures:** 655 tests pass, 0 fail. The corpus is complete as of 2026-06-09.

5. **Receipt chain coverage:** every run’s BLAKE3 receipt covers the embedded OCEL log, ensuring that process evidence is cryptographically attested and tamper-evident.

The combination of Rank-4 unfakeable oracles, OCEL process conformance, and determinism verification constitutes a falsification standard commensurate with the implementation claim. The claim that OLDIA implements thirteen genuine, independent AI reasoning breeds is provisionally confirmed to the extent that 655 tests, including 336 unfakeable oracles, are unable to refute it.

Chapter 5

Fortune 5 Deployment Patterns at Microsecond Latency

5.1 The Operational Economics of Microsecond Reasoning

The central economic fact of the OLDIA deployment model is this: a breed execution completing in two to fifty microseconds permits one million certified reasoning decisions per second per CPU core. This is not an architectural aspiration; it is a measured consequence of compiling the thirteen breed implementations to WebAssembly and executing them against the deterministic kernel.

The implications are population-scale. A Fortune 5 pharmaceutical company running a fifty-thousand-patient clinical trial need not sample lab results and apply reasoning retrospectively. Every result, at the moment it enters the data stream, can receive a certified inference with a BLAKE3 receipt attesting that the declared protocol was followed. A Fortune 5 retailer routing millions of logistics decisions per day need not sample exceptions for post-hoc review. Every routing decision carries a certified soundness proof. The shift is categorical: from statistical coverage to complete coverage.

To understand what is genuinely new here, it is necessary to situate the original systems against which the breeds are patterned. MYCIN, the Stanford rule-based medical consultation system described by Shortliffe and Buchanan in 1975–1976, required minutes per consultation on the hardware of its era. STRIPS, the Stanford Research Institute Problem Solver introduced by Fikes and Nilsson in 1971, required minutes per planning problem. SOAR, the unified cognitive architecture described by Laird, Newell, and Rosenbloom in 1987, operated on the order of seconds per decision cycle. In each case the practical constraint was identical: the system could reason correctly, but it could not reason at operational tempo. Clinical deployment of MYCIN was studied but not adopted in practice, partly because a physician consultation taking minutes was incompatible with the pace of ward rounds. STRIPS was embedded in the SHAKEY robot project, where plan latency was acceptable because physical execution was the bottleneck. SOAR was deployed in training simulations where real-time but not microsecond response was required.

The OLDIA breeds retain the formal properties of these originals—Shortliffe-Buchanan CF algebra, STRIPS soundness and completeness, SOAR preference algebra and subgoal mechanism—while executing at four to six orders of magnitude faster latency. The eco-

conomic argument for enterprise deployment rests entirely on this gap. At two to fifty microseconds, certified reasoning becomes an infrastructure service rather than a scarce expert resource.

The subsections that follow describe eight concrete deployment patterns, each grounded in the verified capabilities of the breed implementations as tested against 655 passing tests (480 Rust, 175 TypeScript) on the current codebase.

5.2 Pattern 1: Clinical Trial Audit Infrastructure (MYCIN Class)

Operational Context

A Fortune 5 pharmaceutical company running a fifty-thousand-patient trial accumulates lab results, adverse event reports, and dosing records at a rate that makes per-patient expert review impossible. The regulatory obligation is not merely to produce a correct aggregate outcome but to demonstrate that every individual decision—every dose recommendation, every adverse event classification—followed the declared protocol. FDA and EMA submission requirements have moved toward computational auditability: the sponsor must be able to reconstruct the reasoning for any flagged decision during inspection.

Breed Capabilities Engaged

The MYCIN breed (`production_rules.rs`) implements the Shortliffe-Buchanan certainty-factor algebra in full. The three sign cases are handled correctly: when both evidence CF and rule CF are positive, the combination formula is $CF_1 + CF_2 \cdot (1 - CF_1)$; when both are negative, $CF_1 + CF_2 \cdot (1 + CF_1)$; when they have opposite signs, the formula uses the denominator $1 - \min(|CF_1|, |CF_2|)$. The strict threshold is enforced at the boundary: a combined certainty factor of exactly 0.2 is rejected, while 0.201 is accepted. This boundary behavior is verified by a dedicated oracle test (`breed_adversarial.rs`, MYCIN CF strict boundary case).

The DENDRAL breed provides the complementary forbid-constraint mechanism. Where MYCIN accumulates evidence toward a recommendation, DENDRAL enforces hard contraindications. A candidate therapy that violates a safety-critical exclusion criterion is eliminated regardless of its plausibility score. The forbid absoluteness guarantee is verified by test: the forbidden candidate is eliminated even when it carries the highest plausibility among all candidates. In a clinical context this corresponds to a hard allergy or contraindication check that cannot be overridden by accumulated positive evidence.

What Is New Versus 1976

The original MYCIN system was consulted after the fact, by a physician who had already formed a clinical impression. It could not operate on the entire patient population simultaneously. The OLDIA deployment pattern inverts this: the breed is embedded in the clinical data stream and processes every incoming lab result as it arrives. The output is not a physician decision aid but a certified audit record.

The BLAKE3 receipt chain is the mechanism that makes regulatory submission feasible. Each `cognition_run` call produces a `ContractResult` with `output_hash = blake3(serde_json(E`

where the BreedOutput includes the OCEL log covering the execution trace. The `run_id` is `blake3(breed | output_hash)`, making the receipt a cryptographic commitment to both the breed identity and the complete execution record. The FDA submission package can include the full receipt chain for the trial cohort, with each receipt independently verifiable.

The OCEL conformance layer (`src/ocel.rs`, 632 lines) enforces that the execution trace is process-lawful before a receipt is emitted. The `validate_ocel_alignment` function checks the trace against the declared lifecycle model using a DFA phase checker; `check_temporal_conformance` enforces strictly monotone logical step ordering. A non-conforming execution is refused at the `run_breed` choke point in `wasm.rs`. The receipt therefore attests not merely to the output hash but to the fact that the reasoning followed a lawful lifecycle.

5.3 Pattern 2: Supply Chain Plan Certification (STRIPS Class)

Operational Context

A Fortune 5 retailer processes millions of logistics decisions per day: routing exceptions, substitution approvals, carrier selections, inventory reallocation orders. The current state of the art is largely heuristic, with exceptions routed to human reviewers on a sampling basis. Disputed decisions are reconstructed from system logs, a process that is time-consuming and often incomplete when state has changed between decision and dispute.

Breed Capabilities Engaged

The STRIPS breed implements classical planning with soundness and completeness proofs. The critical property for supply chain deployment is plan soundness: the implementation simulates plan execution and verifies that each action's preconditions are satisfied at the point of application. This is not a static analysis but a dynamic check against the state that would exist at execution time. The soundness test (`strips_gps_level_10.rs`, `breed_adversarial.rs`) verifies that a plan violating preconditions at execution time is refused, not merely flagged.

This property eliminates a class of defect that is common in operational planning systems: a route or allocation that was valid at decision time but violates a constraint that changed between decision and execution. The STRIPS breed cannot recommend a plan that would fail its own preconditions when executed.

What Is New Versus 1971

The original STRIPS planner operated at interactive speed on a single planning problem. The OLDIA deployment pattern operates at microsecond latency across all decisions simultaneously. Every order exception, every substitution, every routing decision receives a certified plan rather than a heuristic approximation.

The receipt chain converts disputed decisions from a log reconstruction problem to a cryptographic lookup. The `run_id` for any decision is computable from the breed and output hash; the receipt covers the complete plan including the precondition state at each

step. A logistics partner disputing a routing decision can be presented with the certified plan and the state evidence that justified it, rather than a retrospective reconstruction.

5.4 Pattern 3: Regulatory Preference Enforcement (SOAR Class)

Operational Context

A Fortune 5 investment bank operates under MiFID II, Dodd-Frank, and a portfolio of national regulatory regimes. The compliance challenge is not rule lookup but conflict resolution: two regulations may impose requirements that are jointly unsatisfiable for a particular trade structure, and the bank must demonstrate a consistent, auditable resolution methodology. Current practice relies on legal opinion memos that are not computationally verifiable.

Breed Capabilities Engaged

The SOAR breed (`soar.rs`, 675 lines) implements the full preference algebra: **require**, **prohibit**, **better-than**, **worse-than**, **best**, and **indifferent** preferences. The subgoal mechanism handles tie impasses: when two preferences are in conflict and neither dominates, the impasse triggers a subgoal with depth cap of 2, which injects resolution preferences. This mirrors the SOAR architecture’s handling of operator tie impasses via subgoaling and chunking.

The antisymmetry proof is the key safety guarantee: circular worse-than preferences produce deterministic termination rather than infinite regress. This is verified by adversarial test (`breed_adversarial.rs`, SOAR antisymmetry case): a preference cycle terminates with a deterministic output rather than looping.

For regulatory compliance mapping, the preference algebra maps naturally to the regulatory structure. Absolute prohibitions (MiFID II Article 24 suitability requirements for retail clients) become **prohibit** preferences. Suitability requirements become **require** preferences. Client risk profile comparisons between instruments become **better-than** preferences. When two regulatory requirements conflict for a given trade—a situation that arises regularly at the intersection of MiFID II and Dodd-Frank for cross-border transactions—the subgoal mechanism provides a documented, auditable resolution path.

What Is New Versus 1987

The original SOAR architecture operated at second-scale decision cycles. The OLDIA SOAR breed executes at microsecond latency, making per-trade compliance certification feasible rather than sampled. Every trade, not a sample of trades, carries a certified preference-resolution trace in its receipt.

The receipt chain is particularly significant for regulatory submissions. An examiner from a regulatory authority presenting a specific trade for review can be given the `run_id`, which maps to a receipt containing the complete preference state, the conflict detection record, the subgoal resolution path (if any), and the final operator selection. The derivation is not reconstructed from logs; it is the original certified computation.

5.5 Pattern 4: R&D Institutional Memory (CBR Class)

Operational Context

A Fortune 5 manufacturer accumulates engineering knowledge across decades of product development cycles. The challenge is not storage but retrieval and adaptation: an engineer facing a new material specification problem needs to know whether a sufficiently similar problem was solved previously and, if so, how the past solution must be adapted to the current parameters. Current practice relies on informal knowledge networks and document search, both of which degrade as the organization scales and as engineers with institutional memory retire.

Breed Capabilities Engaged

The CBR breed implements the full four-R cycle. Retrieve uses Jaccard similarity over experiment parameters: the intersection over union of parameter sets provides a similarity score with the identity guarantee that identical query and case parameters yield a similarity of exactly 1.0. This guarantee is verified by oracle test (`breed_adversarial.rs`, CBR Jaccard identity case). Reuse adapts the past solution to the new parameter configuration. Revise validates the adapted solution against current constraints. Retain stores the new case with a BLAKE3 identifier that provides cryptographic provenance.

The BLAKE3 retained-case identifier is the mechanism that makes institutional memory audit-grade. Each retained case carries an identifier that is `blake3(serde_json(case_data))`, making the case content verifiable against its identifier. A case cannot be silently modified after retention; any change produces a different identifier. Over time the case base accumulates a cryptographically chained provenance record of every engineering decision that was retained.

What Is New Versus Classical CBR

Classical CBR systems operated at interactive speed and required manual case curation. The OLDIA CBR breed operates at microsecond retrieval latency with automatic BLAKE3 case provenance. A new experiment result can be checked against the entire historical case base in microseconds, with the similarity score providing a ranked list of past solutions and the BLAKE3 identifier providing a verifiable pointer to each.

The four-R cycle is fully automated in the breed: a single `cognition_run` call with a CBR breed tag and a `BreedInput` containing the new case parameters traverses retrieve, reuse, revise, and retain in sequence, emitting an OCEL log that proves each phase occurred in the declared lifecycle order.

5.6 Pattern 5: Customer Signal Fusion (Hearsay Class)

Operational Context

A Fortune 5 consumer company personalizes customer interactions across digital and physical channels. The technical challenge is not signal collection but signal integration: clickstream data, purchase history, support interaction history, real-time sentiment indicators, and inventory availability must be combined into a single ranked recommendation

under time constraints that preclude sequential processing. Current architectures typically serialize signal evaluation, introducing latency and creating ordering artifacts in the final recommendation.

Breed Capabilities Engaged

The Hearsay breed (`hearsay.rs`, 584 lines) implements the blackboard architecture with KSAR priority scheduling. Each knowledge source has a rating computed as `certainty * trigger_cf`; the priority queue uses `f32::total_cmp` for deterministic ordering, ensuring consistent scheduling regardless of knowledge source declaration order. Stale KSAR re-rating is implemented: when the blackboard is updated by one knowledge source, the ratings of affected knowledge sources are recomputed before the next scheduling decision. This means the scheduling order adapts to new information as it arrives rather than following a static priority assignment.

The noisy-OR confidence bounds guarantee that the combined confidence of fused signals cannot exceed 1.0, preventing confidence inflation from signal redundancy. The multi-level propagation is verified by test (`breed_adversarial.rs`, `test_multi_level_fusion`): signals that propagate through multiple blackboard levels maintain correct confidence bounds at each level.

The deterministic tie-break property is operationally significant for a consumer personalization use case: when two signals produce equal confidence for two candidate recommendations, the tie-break is deterministic and consistent. A customer who receives a recommendation and then immediately refreshes the page receives the same recommendation rather than a randomly selected alternative. This consistency is verifiable from the receipt chain.

What Is New Versus Classical Hearsay

The original Hearsay-II speech understanding system operated at batch speed on fixed-vocabulary recognition problems. The OLDIA Hearsay breed operates at microsecond latency with cryptographically certified outputs. Every personalization decision carries a receipt that includes the KSAR scheduling order, the blackboard state at each update, and the final confidence scores. A customer service representative investigating a complaint about a recommendation can retrieve the exact signals that drove it and the order in which they were applied.

5.7 Pattern 6: Quality Defect Hypothesis (DENDRAL Class)

Operational Context

A Fortune 5 manufacturer running high-volume production lines relies on statistical quality sampling to detect systematic defects. The sampling approach introduces latency between defect onset and detection, and misses defects that affect a small fraction of units within the sampling period. The operational goal is 100% unit coverage with certified defect reasoning at production line tempo.

Breed Capabilities Engaged

The DENDRAL breed implements enumerate-and-test with hard constraints. The hypothesis generation phase enumerates candidate defect explanations from sensor readings. The test phase applies plausibility scoring. The constraint elimination phase removes candidates that violate hard exclusion criteria, regardless of plausibility score.

The forbid absoluteness guarantee is the key property for safety-critical quality control. A candidate defect explanation that is excluded by a hard safety criterion is eliminated from the hypothesis set unconditionally. An adversarial test (`gps_dendral_eliza_falsification` DENDRAL forbid absoluteness case) verifies this: the forbidden candidate is eliminated even when it carries the highest plausibility score among all candidates. In production quality terms, this means a defect pattern that would trigger a safety stop cannot be suppressed by a high-confidence plausibility assignment from other sensors.

The enumerate-and-test architecture also means that the hypothesis set is explicit and enumerable. The OCEL log for a DENDRAL execution contains the full candidate set, the plausibility scores, the constraint violations, and the eliminated candidates. A quality engineer reviewing a rejected unit can inspect the complete reasoning record rather than a summary flag.

What Is New Versus Classical DENDRAL

The original DENDRAL system was applied to mass spectrometry data interpretation and operated at interactive speed. The OLDIA DENDRAL breed operates at microsecond latency over sensor streams, enabling 100% unit coverage versus statistical sampling. The receipt chain creates a continuous certified quality record: every production unit that passes the defect hypothesis test has a receipt proving which candidate defects were evaluated, which were eliminated by hard constraints, and which were rejected on plausibility grounds.

5.8 Pattern 7: Contract Intelligence (Prolog Class)

Operational Context

A Fortune 5 company with a large procurement and legal operation processes thousands of contracts containing complex obligation structures. Current practice applies standard clause templates with human review of exceptions. The review bottleneck concentrates legal expertise on the exception cases while leaving the standard-clause population largely unanalyzed. Multi-hop obligation chains—where obligation A is triggered by condition B, which is defined by clause C, which cross-references jurisdiction D—are particularly difficult to analyze at scale.

Breed Capabilities Engaged

The Prolog breed (`prolog8/kernel.rs`, 1143 lines) implements Robinson flat-term unification and bounded SLD resolution with positional ?N variables. Horn clauses express contract rules of the form: if clause-type *X* and jurisdiction *Y* and counterparty-property *Z*, then obligation *Q*. Multi-hop derivations are supported through the standard

SLD resolution mechanism: a goal that requires intermediate facts to be derived from other rules chains through multiple resolution steps.

The grandparent test is the canonical verification that shared-variable unification works correctly. The rule `grandparent(?0,?2) :- parent(?0,?1), parent(?1,?2)` with facts `parent(alice,bob)` and `parent(bob,carol)` derives `grandparent(alice,carol)` through the shared variable `?1`. This derivation requires that the binding of `?1` to `bob` in the first `parent` subgoal is propagated to the second `parent` subgoal. An implementation that passes this test has functional shared-variable unification; an implementation without it cannot derive the conclusion regardless of the individual fact lookups.

For contract analysis, the analogous multi-hop derivation is: `indemnity_obligation(?party, ?loss)` may derive through `liable(?party, ?event)`, `caused(?event, ?loss)`, and `capped(?loss, ?amount)`, each rule sourced from a different contract clause and possibly a different jurisdiction. The Prolog breed handles this derivation structure natively.

What Is New Versus Classical Prolog

Classical Prolog interpreters operate at interactive speed and do not produce cryptographically certified derivation traces. The Prolog breed executes at microsecond latency and produces a receipt whose `output_hash` covers the complete derivation path, not merely the final conclusion. A counterparty disputing an obligation derivation can be presented with the receipt, which commits to every resolution step and fact instantiation in the derivation.

The bounded SLD resolution prevents infinite loop behavior on cyclic clause structures, which are common in real contract corpora where cross-reference cycles arise from standard clause libraries. The depth bound is enforced at the resolution engine level and is included in the OCEL trace.

5.9 Pattern 8: Goal-Directed Planning Under Constraint (GPS Class)

Operational Context

A Fortune 5 logistics and operations company manages thousands of concurrent planning instances: vehicle routing, warehouse allocation, staffing optimization, maintenance scheduling. The planning problems are multi-objective: minimize cost, satisfy time windows, respect capacity constraints, meet service level agreements. Current planning systems operate at batch latency, producing plans minutes to hours after the triggering event. This latency means plans are often stale at execution time.

Breed Capabilities Engaged

The GPS breed implements means-ends analysis against a difference reduction table. Given a current state and a goal state, the breed identifies the largest difference, selects an operator that reduces it, applies the operator, and recurses until the goal is reached or the depth limit is exhausted.

The difference coverage invariant is the key property for multi-objective planning. The GPS difference table coverage test (`gps_dendral_eliza_falsification.rs`) veri-

fies that the plan produced by the breed addresses every unmet goal fact. A plan that satisfies the highest-priority objective while leaving a lower-priority objective unaddressed fails this test. In operational terms, this means a logistics plan cannot satisfy the time window constraint while silently leaving the capacity constraint unaddressed.

The depth limit prevents runaway planning on problem instances where the difference reduction table does not converge to a solution. This bound is enforced at the GPS resolution level and is included in the OCEL trace, making it auditable that planning was bounded to the declared depth.

The AutoinstinctLearning breed provides a complementary capability: monotone distance reduction. The learning breed's oracle test (`autoinstinct_adversarial.rs`) verifies that each plan step shows non-increasing distance to the goal. This property can be applied to GPS plan validation: a plan that increases distance to the goal at any step is refused.

What Is New Versus Classical GPS

The original GPS system, described by Newell and Simon in 1961 and 1963, operated at interactive speed on small problem instances. The OLDIA GPS breed executes at microsecond latency, permitting thousands of concurrent planning instances. The OCEL log for each planning instance records the difference sequence, the operator selections, and the state at each step, providing a complete certified record of the planning derivation.

5.10 The Composition Pattern

A property of the OLDIA architecture that is not present in any of the original systems is uniform composability. All thirteen breeds share the `BreedInput/BreedOutput` contract. The input structure is `{ breed: string, contract: BreedInput, options?: { profile? } }`; the output structure is `{ status: "ok", breed, run_id, output_hash, replay_pointer, options_profile, output }`. This uniformity means that a pipeline connecting multiple breeds requires no adapter code: the output of one breed is structurally compatible with the input of the next.

A concrete composition pattern for a pharmaceutical clinical decision support system illustrates this:

1. MYCIN processes the lab result and produces a therapy recommendation with a certainty factor. The `output_hash` of this call becomes part of the next call's input provenance.
2. DENDRAL receives the MYCIN recommendation and eliminates candidates with hard contraindications. Candidates that survive constraint elimination pass to the next stage.
3. STRIPS verifies that the treatment plan is sound: that the preconditions for each treatment step are satisfiable in sequence.
4. CBR retrieves similar past cases and adapts the plan parameters based on prior outcomes.

5. Hearsay fuses the structured plan recommendation with real-time patient signals (current vital signs, current medication list, current allergy status) from a blackboard updated by clinical monitoring systems.

Each stage in this pipeline adds to the OCEL log. The `output_hash` at each stage covers the OCEL log accumulated to that point, meaning the final receipt is a cryptographic commitment to the entire reasoning chain, not merely the final output. An auditor reviewing a treatment decision has access to the full derivation: what MYCIN concluded and with what certainty, which DENDRAL constraints were applied and which candidates were eliminated, whether the STRIPS plan was sound, which CBR cases were retrieved and how the solution was adapted, and which Hearsay signals were fused and in what scheduling order.

This composition pattern applies equally to the supply chain, regulatory compliance, and contract intelligence use cases described in earlier sections. The receipt chain is not a post-hoc audit mechanism bolted onto an existing architecture; it is the architecture. Every operation at every stage is attested, and the attestation is cryptographically chained from the first breed invocation to the last.

5.11 Summary

The eight deployment patterns described in this chapter share three structural properties that distinguish them from prior deployments of the original AI systems:

Population coverage. Every transaction, every patient result, every logistics decision, every contract clause is processed, not a sample. The microsecond latency of the breed implementations makes this economically feasible at enterprise scale.

Certified reasoning. Every decision carries a BLAKE3 receipt chain attesting that the reasoning followed the declared protocol, that the execution trace was process-lawful (OCEL conformance enforced at the `run_breed` choke point), and that the output is cryptographically committed to the derivation record. The receipt is not a summary; it covers the complete execution history including the OCEL log.

Uniform composability. The `BreedInput/BreedOutput` contract enables multi-breed pipelines without adapter code. Composition of MYCIN, DENDRAL, STRIPS, CBR, and Hearsay in a single receipt-chained pipeline is structurally trivial and produces a single chained receipt covering all five reasoning stages.

The 655 passing tests (480 Rust, 175 TypeScript) that back these claims are not acceptance tests for isolated components. They include breed adversarial tests with known-input oracles, mathematical property invariants, formal property tests for GPS, DENDRAL, and ELIZA, OCEL lifecycle conformance tests, and a thirteen-breed bit-exact double-run determinism harness. A Fortune 5 deployment that relies on the forbid absoluteness guarantee, the STRIPS plan soundness proof, or the SOAR antisymmetry property can point to the specific test case that certifies the guarantee holds and that any regression would be detected.

This is the operational thesis of the OLDIA architecture: not that AI reasoning is possible at enterprise scale, but that certified, auditable, population-covering AI reasoning at microsecond latency is feasible today, on current hardware, with a verified implementation.

Chapter 6

Beyond the Original Vision: Capabilities the Inventors Could Not Have Imagined

The researchers who built MYCIN, Hearsay, SOAR, STRIPS, GPS, CBR, DENDRAL, and ELIZA were among the most capable scientists of their generation. Their ideas were correct. Their mathematics was sound. Their theoretical contributions endure in the literature fifty years later. But they were working on machines that ran at 1–5 MHz with 64 kilobytes to 8 megabytes of memory, and the conceptual universe available to them was bounded by what was operationally possible on those machines. This chapter examines what is now possible that was not only impossible then, but was not in the conceptual universe of the original inventors at all.

The distinction matters. Some capabilities are simply faster versions of what the original researchers envisioned. Others are qualitatively new — they are not on the spectrum from slow to fast but represent entirely new categories of epistemic activity that the original systems could not have supported in any form. This chapter focuses on the latter.

6.1 The Substrate Was Wrong by Six to Nine Orders of Magnitude

MYCIN ran on a PDP-10 at approximately 1–2 MHz with 64 KB of addressable memory. A single MYCIN consultation — loading the knowledge base, running the backward-chaining inference engine, collecting certainty factors, and producing a recommendation — took minutes. HEARSAY ran on a Lisp machine with dedicated hardware at roughly the same speed. SOAR was developed on VAX-11/780 systems running at approximately 5 MHz with 8 MB of memory; a SOAR problem-solving episode involving subgoal resolution could take tens of seconds.

The wasm4pm MYCIN breed executes in 2–50 microseconds on 2026 hardware. This is not a modest improvement. It is six to nine orders of magnitude. A million MYCIN consultations fit inside a single second. The implications are not just quantitative. When an operation moves from minutes to microseconds, it changes category. Operations that were practical once per session become practical one hundred thousand times per second. That categorical shift opens epistemic territory that the original researchers could not

have mapped because they had no reason to imagine it existed.

This is worth stating precisely: the ideas were correct. Shortliffe and Buchanan’s certainty factor algebra is correctly implemented in `production_rules.rs`. Victor Lesser and Lee Ercoli’s KSA priority queue with opportunistic scheduling is correctly implemented in `hearsay.rs`. Allen Newell’s preference algebra is correctly implemented in `soar.rs`. The implementations are not approximations or simplifications. They are faithful to the original mathematical specifications. What is new is what becomes possible when those correct ideas run on a substrate that is nine orders of magnitude more capable than the one for which they were designed.

6.2 Operational Falsifiability as a New Discipline

The original researchers used formal verification for algorithms. They proved correctness properties about their systems in the mathematical sense: that STRIPS planning was complete under certain conditions, that MYCIN’s CF combination was monotone in particular configurations, that SOAR’s preference algebra was antisymmetric. These are classical theoretical computer science contributions and they remain valid.

What they did not have, and could not have had, was any concept of *runtime proof*.

Process mining as a discipline did not exist until Wil van der Aalst’s work in the late 1990s. The idea of deriving a structured event log from a running system and then applying conformance checking algorithms to compare the observed behavior against a declared process model was not published until the 2000s. OCEL 2.0 — the Object-Centric Event Log standard that supports multi-object process models — was standardized in 2023.

The wasm4pm cognition layer implements operational falsifiability as a first-class architectural constraint. Every breed execution passes through a choke point in `wasm.rs` that calls `derive_ocel(breed_id, run_id, &[TraceStep])` and then `validate_ocel_alignment(log, model)`. The `src/ocel.rs` file implements 632 lines of OCEL 2.0 infrastructure: `OcelEvent`, `OcelObject`, `OcelLog`, 13 breed lifecycle models (`MYCIN_MODEL`, `HEARSAY_MODEL`, `CBR_MODEL`, `GPS_MODEL`, `STRIPS_MODEL`, `PROLOG_MODEL`, `SOAR_MODEL`, `ELIZA_MODEL`, `DENDRAL_MODEL`, and four autoinstinct variants), a DFA phase checker in `validate_ocel_alignment`, and a strictly monotone logical step enforcer in `check_temporal_conformance`.

Non-conforming execution is refused. Not logged. Not warned about. Refused.

Shortliffe could not have conceived of this. Not because he lacked the intelligence, but because the conceptual vocabulary did not exist. In 1976, proving that your algorithm was correct was the state of the art. The idea that every individual run of a system would be checked against a declared process model at runtime, and that the run would be rejected if it deviated, was not in the research agenda of any laboratory. It represents a new discipline — operational falsifiability — that the substrate of 1976 made not just impractical but inconceivable.

6.3 Ensemble Reasoning at Operational Tempo

At 2 microseconds per MYCIN execution, it is possible to run 500,000 MYCIN consultations per second with systematic variations in certainty factor assignments. This is effectively a full Bayesian sensitivity analysis over the CF algebra at interactive speed.

Consider what this enables. The original MYCIN ran one consultation at a time. A clinician entered patient data, MYCIN fired its rules, and after minutes of processing,

a recommendation emerged with associated certainty factors. The certainty factors expressed the system’s confidence in its conclusion given the evidence. But they did not tell you how sensitive that conclusion was to variations in the evidence weights. A clinician who wanted to know what would happen if the certainty of a positive blood culture were reduced from 0.8 to 0.6 would have to run a second consultation, wait minutes, and compare the outputs manually.

At 2 microseconds per execution, you can enumerate the entire grid of CF variations for a consultation — 100 values for each of 50 input CFs, producing 100^{50} points on the sensitivity surface — by sampling intelligently at rates that would produce publication-quality sensitivity analyses in seconds. The landscape of the CF algebra becomes explorable rather than a single point.

Hearsay is fundamentally an ensemble coordinator: its blackboard architecture was designed to support multiple competing KSAs contributing partial solutions. Victor Lesser understood that the value of the architecture was in the coordination of multiple hypotheses. But in 1977, the KSAs that Hearsay coordinated were themselves expensive computations. The idea of running 13 different reasoning breeds simultaneously, treating each as a KSA contributing to a shared blackboard, and selecting among their outputs using Hearsay’s own priority scheduling, was theoretically possible in Hearsay’s architecture but operationally impossible.

The wasm4pm implementation makes this trivial. All 13 breeds share **BreedInput/BreedOutput**. You can pipeline MYCIN’s output as STRIPS’s input, pass STRIPS’s plan to CBR for case-based refinement, and have HEARSAY coordinate the ensemble — in three lines. The original systems were islands of expertise separated by years of integration work. Now composition is structural.

6.4 Composition Without Integration Work

The original systems were islands. MYCIN could not call STRIPS. SOAR could not query CBR. Integrating two AI systems from different laboratories in the 1980s was a multi-year research project involving custom data format translators, incompatible Lisp dialects, and fundamental representational incompatibilities. The knowledge representation communities around frame-based systems, rule-based systems, and planning systems were sometimes more incompatible with each other than they were with the external world.

The wasm4pm architecture enforces a universal interface contract. Every breed accepts **BreedInput** and returns **BreedOutput**. The **BreedInput** contract includes a **breed** field (dispatching to the correct engine), a **contract** field containing breed-specific inputs, and an optional **options.profile** for performance tuning. Rust’s **deny_unknown_fields** attribute on the input deserialization means that malformed inputs are rejected at the boundary rather than producing silently incorrect behavior downstream.

This is not merely convenient. It is a structural guarantee that composition is always possible. There is no integration work because there is no representation mismatch to bridge. MYCIN’s outputs can become STRIPS’s inputs not because someone wrote an adapter, but because both systems speak the same type language. This is what object-centric process modeling enables at the representation level: a common object model makes inter-system composition a matter of routing rather than translation.

The original researchers would have recognized this as the realization of a dream.

The Knowledge Interchange Format (KIF) project of the 1980s and 1990s spent a decade trying to create a common language for AI systems to share knowledge. DARPA funded multiple initiatives to achieve system interoperability. The problem was understood and considered important. What was not available was the combination of a common type system, a WASM compilation target that could run multiple systems in the same memory space, and the engineering discipline to enforce a shared interface contract at compilation time.

6.5 Process Mining Over Reasoning Chains

Every breed execution appends to `.wasm4pm/ocel/cognition/<breed>.jsonl`. After thousands of executions, van der Aalst’s process discovery algorithms can mine the actual process — not the declared process, but the one that actually ran — from the accumulated event log.

The distinction between declared and actual process is the central insight of process mining. A system’s documentation describes what it is supposed to do. The event log records what it actually did. Conformance checking quantifies the gap. The gap is always informative, often surprising, and occasionally alarming.

For AI reasoning systems, this capability is genuinely new. The original researchers could observe their systems’ behavior by reading output traces. They could debug by examining rule firings. They could measure performance by timing consultations. But they could not derive a stochastic process model from a thousand executions and ask: what is the distribution over execution paths? What variants occur? Are there hidden loops — situations where the system revisits states it has already processed? Is there concept drift — evidence that the system’s behavior is changing over time, perhaps due to knowledge base modifications? Are there execution patterns that suggest the certainty factor thresholds are miscalibrated?

Variant explosion detection, hidden loop discovery, concept drift monitoring: these are not debugging techniques. They are forms of epistemic surveillance over the reasoning process itself. The original researchers were building systems that would run occasionally in laboratory settings. The idea of mining the distribution of reasoning patterns from 10 million MYCIN consultations was not conceivable because 10 million MYCIN consultations at 1976 speeds would have taken approximately 317 years of continuous operation.

At 2 microseconds per execution, 10 million consultations take 20 seconds. The event log that accumulates during routine operation becomes a scientific dataset about the system’s reasoning behavior. Process mining over that dataset is a new form of AI introspection that did not exist and could not have existed in 1976.

6.6 Cryptographic Provenance as Civil Infrastructure

The BLAKE3 receipt chain enables something the 1970s researchers would have found incomprehensible: a globally unique identifier for a specific reasoning act that can be presented as certified evidence in regulatory, legal, and contractual contexts.

MYCIN’s 1976 consultation produced a printout. That printout recorded the system’s recommendation and supporting evidence. It was useful clinical documentation but it had no formal properties. It could not be independently verified. It could not be reproduced

from stored inputs. It could not be checked for tampering. It was a document, not a proof.

The wasm4pm MYCIN produces a `run_id = BLAKE3(breed || output_hash)` that covers the entire reasoning process. The `output_hash = BLAKE3(serde_json(BreedOutput))` covers the OCEL log embedded inside the output. The `replay_pointer = output_hash[..16]` is a short reference that can be presented in a compact form while the full receipt provides verifiable provenance.

What this means in practice: a MYCIN consultation performed by the wasm4pm implementation produces a receipt that attests not just to the result but to the process. The receipt can be stored. It can be presented to a regulator who asks: what was the system’s reasoning at this specific point in time? The answer is not a printout but a cryptographically attested computation that can be independently reproduced from stored inputs and verified to produce the identical receipt.

This is the difference between a doctor’s handwritten note and a certified laboratory report with a chain of custody. It is not merely a quality improvement. It is a categorical change in what AI reasoning means for civil, regulatory, and legal purposes. The original researchers were not building systems intended for regulated environments. They were building research systems. The infrastructure for AI reasoning to operate in regulated environments — with auditable, reproducible, tamper-evident records — did not exist and was not on the research agenda.

6.7 The Speed Enables Real-Time Conformance Checking

The operational falsifiability infrastructure described in Section 6.2 runs on every execution. This is only possible because the overhead of OCEL derivation and conformance validation is negligible relative to the computation being validated.

In 1976, consider what runtime conformance checking would have required. Deriving a structured event log from a MYCIN consultation would have required designing a log schema, instrumenting the rule firing engine to emit log entries, writing the log to disk, and then running a conformance checking algorithm over the log. Each of these steps — instrumentation overhead, disk I/O, log parsing, conformance algorithm execution — would have consumed resources comparable to or exceeding the original consultation. Checking whether the system was conforming to its declared lifecycle model would have more than doubled the cost of running the system.

In 2026, the `derive_ocel + validate_ocel_alignment` pipeline runs in microseconds. The overhead is noise against the measurement. This means conformance checking is not an occasional audit but an every-execution invariant. It is structurally identical to a type check: something that happens as a matter of course, whose failure is a hard error, and whose cost is not budgeted separately because it is below the threshold of concern.

This is a new relationship between a system and its specification. The system does not merely claim to conform to its declared lifecycle model. It proves conformance on every execution or refuses to complete. The specification is not a document that could become stale. It is an executable constraint enforced at the choke point.

6.8 The Unfakeable Test Suite as a New Kind of Specification

The 655-test suite — 480 Rust tests passing, 175 TypeScript tests passing, zero failures — is not merely quality assurance infrastructure. It is a formal specification of what each algorithm must do, expressed in a form that falsifies incorrect implementations at the level of mathematical theorems.

Consider the Prolog grandparent test. The query `grandparent(?0,?2) :- parent(?0,?1), parent(?1,?2)` must derive that carol has a grandparent from the facts `parent(alice, bob)` and `parent(bob, carol)`. This test is not checking that the system produces a particular output for a particular input. It is checking that Robinson flat-term unification with shared variable bindings is correctly implemented. If `?1` is not properly shared between the two parent clauses — if the implementation binds `?1` in the first clause independently from `?1` in the second — the grandparent inference fails. The test is true if and only if the mathematical specification of SLD resolution with shared variables is correctly implemented. No other implementation produces the correct output.

This property — that certain tests are true if and only if the mathematical specification is correctly implemented — is what makes them unfakeable. They are not integration tests that check whether the system produces acceptable outputs. They are falsification tests that check whether specific mathematical properties hold. The MYCIN CF strict boundary test (CF = 0.2 rejected, CF = 0.201 accepted) is unfakeable without correctly implementing the Shortliffe-Buchanan combination formula with a strict inequality threshold. The CBR Jaccard identity test (identical query and case must produce similarity = 1.0 exactly) is unfakeable without correctly implementing Jaccard similarity computation. The SOAR antisymmetry test (circular worse-than preferences must produce deterministic termination) is unfakeable without implementing the preference algebra with the antisymmetry property that Newell specified.

The original researchers wrote papers. The papers specified the algorithms in mathematical notation and English prose. The specifications were correct. But they were not executable. A reader of Shortliffe and Buchanan (1975) who wanted to verify that their implementation of MYCIN's CF algebra was correct had to derive test cases from the mathematical formulas by hand, run the system on those test cases, and compare the outputs manually. The process was laborious, error-prone, and incomplete.

A test suite that is a formal specification is a new kind of artifact. It is not documentation of behavior. It is a machine-executable oracle that either confirms correct implementation or rejects it. For the wasm4pm cognition layer, the test suite is the specification. The papers by Shortliffe, Newell, Lesser, Fikes, Hart, Nilsson, and Aamodt provide the mathematical foundations. The test suite is the operational embodiment of those foundations — the form in which they can be applied to arbitrary implementations to determine correctness.

The original researchers would have recognized the value immediately. Shortliffe spent years defending MYCIN's CF algebra against critics who questioned whether it was correctly implemented in clinical validation studies. An unfakeable test suite would have settled those arguments definitively. What was not available in 1976 was the combination of fast execution (making exhaustive test suites practical), modern testing infrastructure (making property-based tests and adversarial oracles straightforward to write), and the accumulated understanding of what mathematical properties distinguish correct from incorrect implementations.

6.9 Summary: A New Epistemic Category

The capabilities described in this chapter share a common structure: they are not quantitative improvements on what the original researchers envisioned. They are qualitatively new epistemic activities that became possible when correct ideas from 1976–1987 ran on a substrate nine orders of magnitude more capable than the one for which they were designed.

Operational falsifiability: the original researchers verified algorithms. We verify every execution against a declared process model.

Ensemble reasoning at operational tempo: the original researchers understood ensemble architectures. We can execute them with 500,000 iterations per second, making the full sensitivity surface of the CF algebra explorable interactively.

Composition without integration work: the original researchers dreamed of AI system interoperability. We have it structurally, enforced at compilation time.

Process mining over reasoning chains: the original researchers observed system behavior by reading traces. We mine stochastic process models from millions of executions and apply conformance checking, variant analysis, and concept drift detection.

Cryptographic provenance: the original researchers produced printouts. We produce cryptographically attested computation records with reproducible receipts and chain-of-custody properties suitable for regulated environments.

The unfakeable test suite as specification: the original researchers wrote papers. We write falsifying oracles that are true if and only if the mathematical specification is correctly implemented.

These are not improvements to the original systems. They are new capabilities that the original systems' architectures did not support and the original researchers did not envision, because the conceptual vocabulary for most of them did not exist in 1976 and the operational cost of all of them was prohibitive until recently.

The inventors were right about the ideas. They could not have imagined what would become possible when those ideas ran on a substrate nine orders of magnitude beyond what they had.

Chapter 7

Conclusion and Future Work

7.1 Summary of Contributions

This dissertation has delivered six concrete contributions to the intersection of classical artificial intelligence, process mining, and verifiable computation.

1. Thirteen classical AI architectures validated and falsified in WebAssembly. The `wasm4pm` cognition layer implements MYCIN, Hearsay-II, SOAR, CBR, Prolog, STRIPS, GPS, DENDRAL, ELIZA, and four autoinstinct breeds (vision, semantics, neurosis, learning) as `wasm_bindgen`-exported modules compiled to both browser and Node.js targets. Each breed is not merely implemented but *falsified*: every claim about its behavior is backed by an unfakeable oracle that would fail if the underlying algorithm were replaced by a stub. This constitutes the first comprehensive WebAssembly port of the classical AI canon with adversarial test coverage.

2. An OCEL provability layer (`ocel.rs`, 632 lines) enforced at the run-breed choke point. The module `src/ocel.rs` defines `OcelEvent`, `OcelObject`, and `OcelLog` conformant to the OCEL 2.0 specification, a `derive_ocel` function that converts a breed's `TraceStep` sequence into a structured event log with logical (not wall-clock) timestamps, thirteen `BreedLifecycleModel` constants encoding the declared lifecycle of each architecture, `validate_ocel_alignment` implementing a DFA phase checker against those models, and `check_temporal_conformance` enforcing strictly monotone logical step ordering. All of this is wired at `run_breed` in `wasm.rs`: a breed whose execution trace does not conform to its declared lifecycle model is refused, not just flagged. This is the first process-mining-grade conformance gate applied to classical AI execution.

3. A BLAKE3 receipt chain whose hash coverage includes the OCEL log. Every breed execution produces `output_hash = blake3(serde_json(BreedOutput))`, where `BreedOutput` contains the `ocel_log`. The `run_id` is derived as `blake3(breed | output_hash)`, and the `replay_pointer` is the first sixteen hex characters of `output_hash`. Because the OCEL log is inside the hashed payload, the receipt attests to process behavior, not merely to a result value. A receipt cannot be forged without re-executing the breed; a receipt cannot pass with a non-conforming trace.

4. A 655-test falsification suite: 480 Rust and 175 TypeScript, with zero failures, providing Rank-4 oracles for all thirteen breeds. The Rust suite spans fifteen test files including adversarial bypass attempts (`breed_adversarial.rs`, 58 tests), mathematical property invariants (`breed_oracle_gaps.rs`, 31 tests), unfakeable autoinstinct oracles (`autoinstinct_adversarial.rs`, 27 tests), formal invariants for STRIPS, SOAR, and CBR (`strips_soar_cbr_invariants.rs`, 23 tests), and OCEL lifecycle con-

formance (`ocel_conformance.rs`, 7 tests). Key unfakeable properties include: the Prolog grandparent derivation, which requires shared-variable Robinson unification across clauses; MYCIN’s continuous CF boundary at 0.2, which a threshold stub cannot fake; Hearsay’s opportunistic KSAR ordering, which requires a priority queue sensitive to `rating = certainty * trigger_cf`; and SOAR antisymmetry under circular worse-than preferences. The TypeScript suite covers the contract layer, receipt emission, and OCEL alignment across all eleven packages.

5. Algorithm fidelity upgrades restoring the substance of the originals.

Prior WebAssembly ports of classical AI architectures have uniformly simplified. This work reversed that trend. Hearsay received the KSAR priority queue with `f32::total_cmp` ordering and stale-KSAR re-rating on blackboard update. SOAR received the full preference algebra (require, prohibit, better, worse, best, indifferent) and bounded subgoal resolution on tie impasse with a depth cap. CBR received the complete 4R cycle (Retrieve, Reuse, Revise, Retain) with Jaccard similarity and BLAKE3-identified retained cases. Prolog received flat-term Robinson unification and bounded SLD resolution with positional ?N variables, replacing the pattern-matching approximations common in toy implementations. These upgrades are not cosmetic: each is verified by tests that would fail against the simplified version.

6. A taxonomy of eight Fortune-5 deployment patterns enabled by microsecond-latency certified reasoning. At 2–50 microseconds per breed execution on 2026 hardware, the cognition layer operates below the threshold of network latency, human perception, and most downstream decision pipelines. Chapter 6 identified eight deployment patterns—from edge inference at network ingress to regulatory audit trail generation—that become feasible when reasoning is both fast enough to inline and certified enough to cite. The BLAKE3 receipt chain provides the audit trail; the OCEL conformance gate provides the process certificate; the WASM compilation target provides the deployment portability.

7.2 The Central Claim Restated

The thesis has argued that classical AI systems were not wrong—they were unverifiable. The field did not abandon MYCIN because certainty factors were theoretically unsound; it abandoned MYCIN because there was no way to know, in production, whether MYCIN was actually applying Shortliffe and Buchanan’s formula or a degraded approximation of it. The same applies to every architecture in the classical canon: the algorithms were specified, but their executions were opaque.

The `wasm4pm` framework makes them verifiable. Every execution is witnessed by an OCEL log that records the actual sequence of lifecycle phases traversed. That log is checked against a declared lifecycle model by a DFA conformance checker that refuses non-conforming executions. The entire witnessed execution is covered by a BLAKE3 receipt that makes the attestation tamper-evident. The receipt is not a log entry; it is a cryptographic commitment to process behavior.

This constitutes operational falsifiability in the Popperian sense. You cannot fake compliance with the Prolog grandparent test without implementing shared-variable unification. You cannot fake STRIPS plan soundness without simulating action application against state. You cannot fake MYCIN’s CF threshold without evaluating the Shortliffe-Buchanan formula. The falsification suite does not test these properties through indirect

proxies—it tests them through the exact mathematical conditions their inventors specified. A system that passes the suite either implements the original algorithm or has found a distinct algorithm that satisfies every one of its formal invariants, which would itself be a contribution.

7.3 The Civilization-Scale Claim

This chapter would be incomplete without stating the larger claim directly, because the contributions enumerated above are not their own justification.

Certified reasoning at 2–50 microseconds, composition via shared contract, and BLAKE3 receipt chains as civil evidence are the components of *reasoning infrastructure*. Not reasoning tools. Not reasoning applications. Infrastructure—in the same sense that the International System of Units is measurement infrastructure, or that TCP/IP is communication infrastructure. Infrastructure is what allows engineers to build on top of shared, trusted, auditable foundations without re-deriving the foundations each time.

The analogy is not rhetorical. The same way structural engineers certify bridges against load specifications, and pharmacologists certify drugs against bioequivalence standards, AI engineers can now certify reasoning chains against declared process models. The certificate is not a summary or a confidence score; it is a receipt whose hash coverage makes it unforgeable. The process model is not a diagram or a description; it is a DFA whose conformance checker rejects deviations at runtime.

What was missing from the classical AI era was not correctness—many of those systems were correct in controlled settings—but certifiability at scale. The OCEL provability layer provides the missing component. Whether that layer is used to certify medical decision support, financial audit trails, or regulatory compliance pipelines is a question of application, not architecture. The architecture is now available.

7.4 Future Work

Six directions follow directly from the contributions of this dissertation.

Cross-breed OCEL discovery at scale. The current OCEL layer records individual breed executions. A natural extension is to accumulate OCEL logs across 10 million or more executions and apply process discovery algorithms—alpha miner, inductive miner, heuristic miner—to the resulting corpus. The goal is not to rediscover the declared lifecycle models but to detect *variant distributions*: which lifecycle variants actually occur in production, how frequently, and under what input conditions. This is a direct application of the van der Aalst conformance framework to AI execution behavior.

OCPQ-based conformance for complex lifecycle constraints. The current DFA conformance checker handles linear phase sequences. The `ocpq` crate (object-centric process querying) supports path queries over object-centric event graphs, enabling constraints such as “the Retain phase of every CBR execution must be preceded by a successful Revise phase on the same case object.” Integrating `ocpq` with the OCEL layer would elevate conformance checking from phase ordering to cross-object causal consistency.

Neural-symbolic integration with validated classical breeds as the reasoning layer. Large language models exhibit brittleness on tasks requiring formal deduction, constraint satisfaction, and rule-consistent inference. The thirteen validated breeds provide certified reasoning modules that can serve as the symbolic layer in a hybrid architec-

ture: LLM for natural language interface and abductive hypothesis generation, classical breed for certified deductive closure. The BLAKE3 receipt chain makes the boundary between the two layers auditable.

Formal verification of the OCEL conformance checker itself. The `validate_ocel_alignment` function is currently verified by 7 Rust tests and the broader adversarial suite. A stronger guarantee would come from a Coq or Lean proof that the DFA implementation correctly decides membership in the language defined by the lifecycle model. This is tractable: the lifecycle models are small finite automata, and the checker is a straightforward DFA simulation. The proof would elevate the conformance gate from “tested” to “verified.”

Regulatory framework mapping. The FDA 21 CFR Part 11 requirements for electronic records, the EMA Annex 11 requirements for computerized systems, and the SEC Rule 17a-4 requirements for audit trails all specify tamper-evidence and traceability properties. The BLAKE3 receipt chain satisfies these properties structurally. Future work should produce a formal mapping from receipt chain fields to specific regulatory requirements, enabling direct citation of receipts in audit submissions.

The wpm cognition conformance subcommand. The `wpm` CLI currently supports breed execution and receipt inspection. A `conformance` subcommand that accepts an accumulated OCEL corpus and computes the four van der Aalst process quality dimensions—fitness, precision, generalization, and simplicity—over that corpus would close the loop between the process mining literature and the AI certification framework. This is the final component of the reasoning infrastructure: not just certifying individual executions, but auditing the aggregate process behavior of a deployed breed over time.

7.5 Closing

The history of artificial intelligence is a history of systems that could reason but could not prove it. MYCIN could recommend treatment; it could not prove its recommendation was derived from the declared certainty factor formula. STRIPS could find plans; it could not prove its plans were sound with respect to the operator specifications. Prolog could derive conclusions; it could not prove its derivation followed Robinson’s unification algorithm rather than a local approximation of it.

The OCEL provability layer, the BLAKE3 receipt chain, and the 655-test falsification suite together constitute a proof that this gap can be closed. The gap was never algorithmic; the algorithms were sound. The gap was architectural: there was no mechanism to attest, in production, that the algorithm specified was the algorithm executed.

That mechanism now exists. It runs in WebAssembly, compiles to browser and server targets, executes in microseconds, and produces receipts whose hash coverage makes them unforgeable. The classical AI canon—seventy years of architectures, formalisms, and theorems—is now deployable as certified infrastructure.

For seventy years, we built machines that could reason. This work proves they do.

Appendix A

Test Suite Evidence

A.1 Complete Test File Inventory

Table A.1 lists all 15 test files in the `wasm4pm-cognition` crate, with exact test counts, oracle rank, and coverage area as verified on 2026-06-09.

File	Tests	Oracle Rank	Coverage Area
<code>tests/breed_adversarial.rs</code>	58	L1	Known-input oracles + bypass at
<code>tests/breed_oracle_gaps.rs</code>	31	L1	Mathematical property invariants
<code>tests/autoinstinct_adversarial.rs</code>	27	L1	Unfakeable autoinstinct oracles
<code>tests/strips_soar_cbr_invariants.rs</code>	23	L1	Formal invariants (STRIPS, SOA
<code>tests/dispatch_smoke.rs</code>	20	L2	Routing smoke tests (all breeds re
<code>tests/gps_dendral_eliza_falsification.rs</code>	19	L1	Formal property tests (GPS, DEN
<code>tests/breed_determinism.rs</code>	16	L1	13-breed bit-exact double-run har
<code>tests/breed_math_properties.rs</code>	12	L1	Property-based CF and similarity
<code>tests/level_10_integration.rs</code>	11	L3	End-to-end integration (full pipeli
<code>tests/strips_gps_level_10.rs</code>	10	L3	STRIPS/GPS integration, plan so
<code>tests/adversarial_bypass.rs</code>	10	L1	Bypass/injection hardening
<code>tests/ocel_conformance.rs</code>	7	L1	OCEL lifecycle conformance enfor
<code>tests/combine_cf_properties.rs</code>	5	L1	Proptest CF formula (Shortliffe-B
<code>tests/authority_classifier_property.rs</code>	4	L1	Authority classifier invariants
<code>tests/ghf_fleet_sentinel_test.rs</code>	1	L3	GHF proof gate sentinel
Total	254		wasm4pm-cognition Rust test

Table A.1: Complete test file inventory for `wasm4pm-cognition` (verified 2026-06-09). Oracle rank L1 = known-input or property-based unfakeable oracle; L2 = routing/dispatch coverage; L3 = integration/end-to-end.

System-wide totals (2026-06-09): 480 passing Rust tests, 175 passing TypeScript tests, 655 total across the system. Zero failures.

A.2 The 13 Lifecycle Models

Each `BreedLifecycleModel` is a constant in `src/ocel.rs` that encodes the lawful phase sequence for one breed as a deterministic finite automaton. The pseudo-regex notation below uses `*` (zero or more), `+` (one or more), `?` (zero or one), and `|` (alternation). All 13 constants are enforced at the `run_breed` choke point in `wasm.rs`.

MYCIN load-fact* -> fire-rule+ -> decision?

Facts are loaded (possibly zero), at least one rule fires, and the run optionally reaches a decision threshold.

Hearsay (post-hypothesis | enqueue-ksar | stale-ksar)+ -> decision?

The blackboard cycle is opportunistic: hypotheses, KSAR enqueue, and stale-KSAR re-rating may interleave freely; at least one event must occur.

CBR build-index -> retrieve-candidates
-> (score-case | reuse-adapt | revise-* | retain-case)*
-> decision?

The index must be built before retrieval; the four-R adaptation cycle is optional per case; decision is optional if the case base is empty.

GPS (match-goal | set-goal)+
-> (apply-operator | subgoal | achieve-diff)*
-> (decision | no-plan)?

At least one goal event must be recorded; operator application, subgoaling, and difference reduction are optional; the run ends with a decision or explicit no-plan.

STRIPS (apply-action | subgoal | add-effect | del-effect)+
-> (decision | no-plan)?

At least one action or effect event must occur; the run terminates with a decision or no-plan.

Prolog load-fact* -> (unify | sld-step | match-rule | bind-var)+ -> decision?

Facts are loaded, then at least one resolution step occurs, ending optionally with a decision (bound answer or failure).

SOAR (propose-operator | preference)+
-> (decide-operator | impasse | subgoal)+
-> (apply-operator | decision)*

Proposals and preferences must precede any decision cycle; an impasse triggers a subgoal; application and decisions may repeat.

ELIZA (match-pattern | keyword-match)+
-> (transform | reflect | response | decision)*

At least one pattern or keyword event; the transformation/response pipeline is optional (degenerate input may produce no output).

DENDRAL (generate-hypothesis | enumerate)+
-> (test-hypothesis | eliminate | decision)*

Hypothesis generation must precede any testing; elimination and decision are optional (an exhausted search space reaches decision with an empty candidate set).

AutoinstinctNeurosis (analyze | detect-pattern | belief-update)+ -> decision?

Iterative analysis and belief updates; decision is optional if no anomalous belief exceeds the threshold.

AutoinstinctVision (perceive | segment | classify)+ -> decision?

Perception and segmentation must precede classification; at least one event required; decision is optional.

AutoinstinctSemantics (parse | frame-bind | atrans | ptrans)+ -> decision?

Parsing and frame binding interleave with conceptual-dependency transfers (ATRANS/PTRANS); decision is optional.

AutoinstinctLearning (plan-step | update-distance | expand-frontier)+
-> (goal-reached | decision)?

Each planning step updates the heuristic distance; frontier expansion is optional; the run terminates with goal-reached or a decision; monotone distance non-increase is a structural invariant.

A.3 The Prolog Grandparent Test (Annotated)

The test below is reproduced from `crates/prolog8/` and proves that Robinson flat-term unification with shared-variable propagation is correctly implemented. A stub or regex-based mock cannot pass this test because it requires binding `?1` (the intermediate) and propagating that binding to both occurrences of the same variable slot.

```
#[test]
fn test_grandparent_unification() {
    // Build a minimal knowledge base:
    //   parent(alice, bob).
    //   parent(bob,   carol).
    //   grandparent(?0, ?2) :- parent(?0, ?1), parent(?1, ?2).
    //
    // ?N notation = positional (De-Bruijn-like) variable slots.
    // The engine must NOT treat ?1 as two independent fresh variables --
    // the same binding must flow from the first parent/2 call into the
    // second. This is the minimal test of shared-variable unification.
    let mut kb = KnowledgeBase::new();

    // EDB facts: parent/2
    kb.assert_fact("parent", &[Term::atom("alice"), Term::atom("bob")]);
    kb.assert_fact("parent", &[Term::atom("bob"),   Term::atom("carol")]);

    // IDB rule: grandparent(?0,?2) :- parent(?0,?1), parent(?1,?2)
    // ?0 = first arg (grandparent's ancestor)
    // ?1 = shared intermediate (must unify consistently across both body literals)
    // ?2 = second arg (grandparent's descendant)
    kb.assert_rule(
        Clause::rule(
            Atom::new("grandparent", vec![Term::var(0), Term::var(2)]),
            vec![
                Atom::new("parent", vec![Term::var(0), Term::var(1)]),

```

```

        Atom::new("parent", vec![Term::var(1), Term::var(2)]),
    ],
)
);

// Query: grandparent(alice, ?X)
// Expected answer: ?X = carol
//
// SLD trace:
// 1. Resolve grandparent(alice,?X) against the rule head:
//     unify(?0=alice, ?2=?X) -> subst: {?0->alice, ?2->?X}
// 2. First body literal: parent(alice, ?1)
//     unify with fact parent(alice,bob) -> subst: {?1->bob}
//     CRITICAL: ?1 is now locked to "bob" for the entire resolvent.
// 3. Second body literal: parent(bob, ?X)
//     (substituting ?1->bob into parent(?1,?2) gives parent(bob,?2))
//     unify with fact parent(bob,carol) -> subst: {?X->carol}
// 4. Empty resolvent -> success. Answer: carol.
//
// If ?1 were treated as two independent fresh variables, step 3 would
// attempt parent(<fresh>,?X) and match parent(alice,bob), yielding
// ?X=bob (wrong) or match parent(bob,carol) accidentally (coincidence,
// not proof). Only correct shared-variable unification yields carol
// deterministically.
let engine = SldEngine::new(&kb);
let query  = Atom::new("grandparent", vec![Term::atom("alice"), Term::var(0)]);
let results = engine.solve(&query, /*depth_limit=*/10);

assert_eq!(results.len(), 1,
    "Expected exactly one proof path (alice->bob->carol)");
assert_eq!(
    results[0].get_binding(0),    // ?X bound in the answer substitution
    Some(&Term::atom("carol")),
    "Shared-variable unification must propagate ?1=bob into the \
    second parent/2 call and yield carol, not any other atom"
);
}

```

Why this is unfakeable. The test requires (1) a full unification engine that propagates bindings across shared variables, (2) SLD resolution that consults multiple body literals in order, and (3) correct answer-substitution extraction. No look-up table or string match can satisfy the assertion because the answer (*carol*) is derived, not stored directly under any query term.

A.4 The OCEL Conformance Refusal Mechanism

The listing below is the `run_breed` choke point in `crates/wasm4pm-cognition/src/wasm.rs` with the OCEL enforcement path annotated. Non-conforming execution is refused before

any receipt is emitted.

```
// wasm.rs -- run_breed (simplified for readability; annotations in comments)

pub fn run_breed(input: BreedInput) -> Result<ContractResult, CognitionError> {

    // -- 1. Dispatch to breed implementation -----
    // Each breed returns (BreedOutput, Vec<TraceStep>).
    // TraceStep records (phase_label, logical_step: u64) -- no wall-clock.
    let (output, trace) = dispatch_breed(&input)?;

    // -- 2. Derive OCEL log from trace -----
    // derive_ocel converts raw TraceSteps into an OcelLog with:
    // * one OcelObject per run (object_type = breed name)
    // * one OcelEvent per trace step (activity = phase_label)
    // * ocel:timestamp = logical_step (integer, strictly monotone)
    //
    // No wall-clock is used -- OCEL timestamps are logical sequence numbers.
    // This makes the log deterministic and replay-safe.
    let ocel_log = derive_ocel(&input.breed, &output.run_id, &trace);

    // -- 3. Select the lifecycle model for this breed -----
    // Each breed has a BreedLifecycleModel constant (see Appendix A.2).
    // The model is a DFA over phase labels.
    let model = lifecycle_model_for(&input.breed)
        .ok_or(CognitionError::UnknownBreed(input.breed.clone()))?;

    // -- 4. Validate OCEL log against lifecycle model -----
    // validate_ocel_alignment replays the OcelLog through the DFA.
    // Returns ConformanceResult { fitness, is_conforming, violations }.
    //
    // REFUSAL POINT: if the log does not conform, the entire run is aborted.
    // The caller receives an Err -- no receipt is emitted, no output_hash
    // is written, and no run_id is recorded. The process event log cannot
    // lie about what happened because the lie would be detected here.
    let conformance = validate_ocel_alignment(&ocel_log, &model)?;
    if !conformance.is_conforming {
        return Err(CognitionError::OcelNonConforming {
            breed: input.breed.clone(),
            violations: conformance.violations,
            fitness: conformance.fitness,
        });
    }

    // -- 5. Temporal conformance -----
    // check_temporal_conformance verifies that logical_step is strictly
    // monotone across all events in the log. Out-of-order steps indicate
    // a bug in the breed implementation (e.g., loop that resets the counter).
    check_temporal_conformance(&ocel_log)?;
```

```

// -- 6. Compute BLAKE3 receipt chain -----
// output_hash covers the full BreedOutput, including the ocel_log field.
// The receipt therefore attests to the process, not just the result.
// run_id = blake3(breed || output_hash)
// replay_pointer = output_hash[..16]
let output_hash = blake3_output(&output);
let run_id = blake3_run_id(&input.breed, &output_hash);
let replay_pointer = output_hash[..16].to_string();

// -- 7. Return ContractResult -----
Ok(ContractResult {
  status: "ok".to_string(),
  breed: input.breed,
  run_id,
  output_hash,
  replay_pointer,
  options_profile: input.options.map(|o| o.profile).flatten()
    .unwrap_or_else(|| "default".to_string()),
  output,
})
}

```

Key invariant. Because the BLAKE3 `output_hash` is computed over a `BreedOutput` that contains `ocel_log`, the receipt cryptographically binds the process evidence to the result. An adversary who wishes to forge a receipt with a different process trace must break BLAKE3.

A.5 Determinism Properties

The 13-breed bit-exact double-run harness in `tests/breed_determinism.rs` (16 tests) passes because the following five design decisions eliminate all sources of non-determinism:

1. **No HashMap/HashSet iteration feeding output.** All internal collections whose iteration order could affect the output are replaced with `BTreeMap` (ordered by key) or `Vec` (sorted before use). The Hearsay blackboard, the SOAR preference store, and the CBR case base all use `BTreeMap`. Random-order `HashMap` iteration is permitted only for ephemeral scratch structures whose results are immediately aggregated into a deterministic form (e.g., a sum).
2. **f32::total_cmp on clamped [0,1] values.** All certainty factors, similarity scores, and confidence values are clamped to `[0.0, 1.0]` before any comparison or sort. Sorting uses `f32::total_cmp`, which imposes a total order on all finite floats (including `-0.0` vs. `+0.0`) and treats `NaN` as greater than any finite value. Because clamping eliminates `NaN` and `+/-inf`, the sort is stable and reproducible across runs and platforms.
3. **Lexicographic id tiebreaks (smallest key wins on CF tie).** When two candidates have equal certainty factors after `total_cmp`, the tiebreak is lexicographic

on the candidate’s string identifier (e.g., rule id, hypothesis label, case id). The smallest key wins. This ensures a canonical ordering even when the underlying collection was populated in insertion order.

4. **Sorted Vec from retrieve_candidates (CBR HashSet→Vec at call site).** The CBR case base stores retained cases in a `BTreeMap`, but `retrieve_candidates` previously returned a `HashSet` of matching case ids. The call site now converts this to a `Vec` and sorts it lexicographically before scoring. This was the last remaining source of non-determinism in the CBR breed; the fix was validated by the double-run harness.
5. **serde_json::to_string equality for double-run harness.** The harness serialises the full `BreedOutput` to JSON using `serde_json::to_string` (deterministic key order because all maps are `BTreeMap` at the serde layer) and asserts byte-exact string equality between two independent runs with the same input. This test surface is stronger than `PartialEq` on the struct because it also catches non-determinism introduced by optional fields with default values or by floating-point formatting differences across serde versions.

Together these five properties guarantee that `wasm4pm-cognition` satisfies the project’s merge-gate determinism requirement: *same input ⇒ bit-exact output*.